

File: ./cea2045/cea2045/util/CEA2045Time.cpp

```
/*  
  
 * CEA2045Time.cpp  
  
 *  
  
 * Created on: Jun 20, 2016  
  
 * Author: dupes  
  
 */  
  
#include <time.h>  
  
#include <string.h>  
  
  
#include "CEA2045Time.h"  
  
  
namespace cea2045  
  
{  
  
  
  
  
  
  
  
  
  
const unsigned int CEA2045Time::C_EPOCH_DIFF = 946684800;  
  
  
  
  
CEA2045Time::CEA2045Time()  
  
{  
  
}  
  
  
  
  
CEA2045Time::~CEA2045Time()  
  
{  
  
}
```

```
unsigned int CEA2045Time::now()
```

```
{
```

```
    return time(NULL) - C_EPOCH_DIFF;
```

```
}
```

```
unsigned int CEA2045Time::mktime(int year, int month, int day, int hours, int minutes,  
int seconds)
```

```
{
```

```
    tm tm_utc;
```

```
    memset(&tm_utc , 0, sizeof(tm));
```

```
    tm_utc.tm_year = year - 1900;
```

```
    tm_utc.tm_mon = month - 1;
```

```
    tm_utc.tm_mday = day;
```

```
    tm_utc.tm_hour = hours;
```

```
    tm_utc.tm_min = minutes;
```

```
    tm_utc.tm_sec = seconds;
```

```
    tm_utc.tm_isdst = -1;
```

```
    return timegm(&tm_utc) - C_EPOCH_DIFF;
```

```
}
```

```
} /* namespace cea2045 */
```

File: ./cea2045/cea2045/util/Checksum.cpp

```
/*  
  
 * CheckSum.cpp  
  
 *  
  
 * Created on: Aug 20, 2015  
  
 * Author: dupes  
  
 */  
  
#include "Checksum.h"  
  
namespace cea2045 {  
  
Checksum::Checksum()  
  
{  
  
}  
  
Checksum::~Checksum()  
  
{  
  
}  
  
bool Checksum::validate(const unsigned char *buffer, unsigned int length)  
  
{  
  
    unsigned short checksum = *(unsigned short *)(buffer + length - 2);
```

```

unsigned short calculatedChecksum = calculate(buffer, length - 2);

return calculatedChecksum == checksum;
}

unsigned short Checksum::calculate(const unsigned char *buffer, unsigned int length)
{
    int checksum1 = 0xaa;

    int checksum2 = 0;

    for (unsigned int i = 0; i < length; i++)
    {
        checksum1 += buffer[i];

        checksum1 = checksum1 % 0xff;

        checksum2 += checksum1;

        checksum2 = checksum2 % 0xff;
    }

    unsigned char value[2];

    value[0] = 255 - ((checksum1 + checksum2) % 255);

    value[1] = 255 - ((checksum1 + value[0]) % 255);

    return *((unsigned short *)&value);
}

```

```
} /* namespace cea2045 */
```

File: ./cea2045/cea2045/util/MSTimer.h

```
/*
```

```
 * MSTimer.h
```

```
 *
```

```
 * Created on: Mar 10, 2015
```

```
 * Author: dupes
```

```
 */
```

```
#ifndef MSTIMER_H_
```

```
#define MSTIMER_H_
```

```
#include <sys/time.h>
```

```
namespace cea2045 {
```

```
class MSTimer
```

```
{
```

```
private:
```

```
    struct timeval m_timerStart;
```

```
    unsigned long int m_durationMS;
```

```
public:
```

```
    MSTimer(unsigned long int durationMS = 0, bool expired = false);
```

```
virtual ~MSTimer();

void reset();

unsigned long int getElapsedMS();

void sleep();

bool isExpired();

};

} // namespace cea2045

#endif /* MSTIMER_H_ */
```

File: ./cea2045/cea2045/util/CEA2045Time.h

```
/*
 * CEA2045Time.h
 *
 * Created on: Jun 20, 2016
 * Author: dupes
 */

#ifndef CEA2045_CEA2045_UTIL_CEA2045TIME_H_
#define CEA2045_CEA2045_UTIL_CEA2045TIME_H_

namespace cea2045
```

```

{

class CEA2045Time

{

private:

    static const unsigned int C_EPOCH_DIFF;


public:

    CEA2045Time();

    virtual ~CEA2045Time();


    static unsigned int now();

    static unsigned int mktime(int year, int month, int day, int hours, int minutes, int
seconds);

};


} /* namespace cea2045 */


#endif /* CEA2045_CEA2045_UTIL_CEA2045TIME_H_ */

```

File: ./cea2045/cea2045/util/Checksum.h

```

/*

* CheckSum.h

*

* Created on: Aug 20, 2015

* Author: dupes

*/

```

```
#ifndef CEA2045_CEA2045_CHECKSUM_H_

#define CEA2045_CEA2045_CHECKSUM_H_

namespace cea2045 {

class Checksum

{

private:

    Checksum();

public:

    virtual ~Checksum();

    static bool validate(const unsigned char *buffer, unsigned int length);

    static unsigned short calculate(const unsigned char *buffer, unsigned int length);

};

} /* namespace cea2045 */

#endif /* CEA2045_CEA2045_CHECKSUM_H_ */
```

File: ./cea2045/cea2045/util/MSTimer.cpp

```
/*

 * MSTimer.cpp

 *
```


* Created on: Mar 10, 2015

* Author: dupes

*/

```
#include "MSTimer.h"
```

```
#include <unistd.h>
```

```
#include <stdio.h>
```

```
namespace cea2045 {
```

```
MSTimer::MSTimer(unsigned long int durationMS, bool expired) :
```

```
    m_durationMS(durationMS)
```

```
{
```

```
    reset();
```

```
    if (expired)
```

```
{
```

```
    m_timerStart.tv_sec = 0;
```

```
    m_timerStart.tv_usec = 0;
```

```
}
```

```
}
```

```
MSTimer::~~MSTimer()
```

```
{
```

```
}
```

```
void MTimer::reset()
```

```
{  
    gettimeofday(&m_timerStart, NULL);  
}
```

```
unsigned long int MTimer::getElapsedMS()
```

```
{  
    struct timeval now;  
  
    gettimeofday(&now, NULL);  
  
    return (now.tv_sec * 1000 + now.tv_usec / 1000) - (m_timerStart.tv_sec * 1000 +  
m_timerStart.tv_usec / 1000);  
}
```

```
void MTimer::sleep()
```

```
{  
    unsigned long int elapsedMS = getElapsedMS();  
  
    if (elapsedMS < m_durationMS)  
    {  
        usleep((m_durationMS - elapsedMS) * 1000);  
    }
```

```
    reset();
```

```
}
```

```
bool MTimer::isExpired()
```

```
{
```

```
    if (getElapsedMS() >= m_durationMS)
```

```
        return true;
```

```
    return false;
```

```
}
```

```
} // namespace cea2045
```

File: ../cea2045/cea2045/device/CEA2045DeviceUCM.h

```
/*
```

```
 * CEA2045DeviceUCM.h
```

```
 *
```

```
 * Created on: Aug 25, 2015
```

```
 * Author: dupes
```

```
*/
```

```
#ifndef CEA2045_CEA2045DEVICEUCM_H_
```

```
#define CEA2045_CEA2045DEVICEUCM_H_
```

```
#include "../processmessage/IUCM.h"
```

```

#include "CEA2045Device.h"

#include "ICEA2045DeviceUCM.h"

#include "message/Basic.h"

#include "message/Intermediate.h"

#include "message/SetEnergyPrice.h"

#include "message/SetSetpoints.h"

#include "message/SetTemperatureOffset.h"

#include "message/StartCycling.h"

#include "message/TerminateCycling.h"


namespace cea2045 {

class CEA2045DeviceUCM : public CEA2045Device, public ICEA2045DeviceUCM
{
public:
    CEA2045DeviceUCM(std::unique_ptr<ILinkLayerComm> linkLyer,
std::unique_ptr<IProcessMessage> processMessage);

    CEA2045DeviceUCM(ILinkLayerComm *linkLyer, IProcessMessage *processMessage);

    virtual ~CEA2045DeviceUCM();

    std::future<ResponseCodes> intermediateGetDeviceInformation();

    std::future<ResponseCodes> intermediateGetCommodity();

    std::future<ResponseCodes> intermediateGetTemperatureOffset();

    std::future<ResponseCodes> intermediateGetSetPoint();

    std::future<ResponseCodes> intermediateGetPresentTemperature();

```

```

std::future<ResponseCodes>      intermediateSetTemperatureOffset(unsigned      char
temperatureOffset, TemperatureUnits units);

std::future<ResponseCodes>      intermediateSetSetpoints(unsigned      short      deviceType,
TemperatureUnits units, unsigned short setpoint1, unsigned short setpoint2);

std::future<ResponseCodes>      intermediateSetEnergyPrice(unsigned      short      currentPrice,
unsigned short currencyCode,
      unsigned char digitsAfterDecimalPoint, unsigned int expirationDatetimeUTC,
      unsigned short nextPrice);

std::future<ResponseCodes>      intermediateStartCycling(unsigned int eventID, unsigned int
startTimeUTCSince2000, unsigned short durationInMinutes,
      unsigned char dutyCycle, unsigned char startRandomizationMinutes, unsigned char
endRandomizationMintues,
      unsigned char criticality);

std::future<ResponseCodes>      intermediateTerminateCycling(unsigned int eventID, unsigned
char endRandomizationInMinutes);


std::future<ResponseCodes>      basicCriticalPeakEvent(unsigned char eventDuration);

std::future<ResponseCodes>      basicEndShed(unsigned char durationToNextEvent);

std::future<ResponseCodes>      basicGridEmergency(unsigned char eventDuration);

std::future<ResponseCodes>      basicLoadUp(unsigned char eventDuration);

std::future<ResponseCodes>
basicOutsideCommConnectionStatus(OutsideCommuncatonStatusCode code);

std::future<ResponseCodes>      basicPresentRelativePrice(unsigned      char
relativePriceIndicator);

std::future<ResponseCodes>      basicNextRelativePrice(unsigned      char
relativePriceIndicator);

std::future<ResponseCodes>      basicQueryOperationalState();

```

```

std::future<ResponseCodes> basicShed(unsigned char eventDuration);

std::future<ResponseCodes> basicPowerLevel(unsigned char powerLevel);


std::future<ResponseCodes> queryMaxPayload();


std::future<ResponseCodes> queryMessageTypeSupported(MessageCode messageCode, unsigned
char messageType1, unsigned char messageType2);


std::future<ResponseCodes> querySupportDataLinkMessages();

std::future<ResponseCodes> querySupportIntermediateMessages();


virtual std::future<ResponseCodes> intermediateSetAdvancedLoadUp(
    unsigned short duration,
    unsigned short value,
    unsigned char units,
    unsigned char suggestedEfficiency = 0xFF,
    unsigned int eventID = 0,
    unsigned int startTime = 0,
    unsigned char startRandomization = 0,
    unsigned char endRandomization = 0) override;


bool start();

void shutDown();

};

} /* namespace cea2045 */

```

```
#endif /* CEA2045_CEA2045DEVICEUCM_H_ */
```

File: ./cea2045/cea2045/device/DeviceFactory.h

```
/*  
  
 * DeviceFactory.h  
  
 *  
  
 * Created on: Aug 25, 2015  
  
 * Author: dupes  
  
 */  
  
  
#ifndef CEA2045_CEA2045_DEVICE_DEVICEFACTORY_H_  
  
#define CEA2045_CEA2045_DEVICE_DEVICEFACTORY_H_  
  
  
#include "CEA2045DeviceUCM.h"  
  
#include "../communicationport/ICommunicationPort.h"  
  
  
namespace cea2045 {  
  
class DeviceFactory  
  
{  
  
    DeviceFactory();  
  
  
public:  
  
    virtual ~DeviceFactory();  
  
  
    static ICEA2045DeviceUCM *createUCM(ICommunicationPort *port, IUCM *ucm);  
  
};
```

```
} /* namespace cea2045 */
```

```
#endif /* CEA2045_CEA2045_DEVICE_DEVICEFACTORY_H_ */
```

File: ./cea2045/cea2045/device/ICEA2045DeviceUCM.h

```
/*
```

```
 * ICEA2045DeviceUCM.h
```

```
 *
```

```
 * Created on: Apr 26, 2016
```

```
 * Author: dupes
```

```
 */
```

```
#ifndef CEA2045_CEA2045_DEVICE_ICEA2045DEVICEUCM_H_
```

```
#define CEA2045_CEA2045_DEVICE_ICEA2045DEVICEUCM_H_
```

```
#include <future>
```

```
namespace cea2045
```

```
{
```

```
class ICEA2045DeviceUCM
```

```
{
```

```
public:
```

```
    ICEA2045DeviceUCM() {};
```

```
    virtual ~ICEA2045DeviceUCM() {};
```



```

virtual bool start() = 0;

virtual void shutDown() = 0;


virtual std::future<ResponseCodes> queryMaxPayload() = 0;


virtual std::future<ResponseCodes> queryMessageTypeSupported(MessageCode messageCode,
unsigned char messageType1, unsigned char messageType2) = 0;


virtual std::future<ResponseCodes> querySupportDataLinkMessages() = 0;

virtual std::future<ResponseCodes> querySupportIntermediateMessages() = 0;


virtual std::future<ResponseCodes> intermediateGetDeviceInformation() = 0;

virtual std::future<ResponseCodes> intermediateGetCommodity() = 0;

virtual std::future<ResponseCodes> intermediateGetTemperatureOffset() = 0;

virtual std::future<ResponseCodes> intermediateGetSetPoint() = 0;

virtual std::future<ResponseCodes> intermediateGetPresentTemperature() = 0;


virtual std::future<ResponseCodes> intermediateSetTemperatureOffset(unsigned char
temperatureOffset, TemperatureUnits units) = 0;

virtual std::future<ResponseCodes> intermediateSetSetpoints(unsigned short deviceType,
TemperatureUnits units, unsigned short setpoint1, unsigned short setpoint2) = 0;

virtual std::future<ResponseCodes> intermediateSetEnergyPrice(unsigned short
currentPrice, unsigned short currencyCode,
    unsigned char digitsAfterDecimalPoint, unsigned int expirationDatetimeUTC,
    unsigned short nextPrice) = 0;

virtual std::future<ResponseCodes> intermediateStartCycling(unsigned int eventID,
unsigned int startTimeUTCsince2000, unsigned short durationInMinutes,

```

```

    unsigned char dutyCycle, unsigned char startRandomizationMinutes, unsigned char
endRandomizationMintues,

    unsigned char criticality) = 0;

virtual std::future<ResponseCodes> intermediateTerminateCycling(unsigned int eventID,
unsigned char endRandomizationInMinutes) = 0;

virtual std::future<ResponseCodes> basicCriticalPeakEvent(unsigned char eventDuration)
= 0;

virtual std::future<ResponseCodes> basicEndShed(unsigned char durationToNextEvent) = 0;
virtual std::future<ResponseCodes> basicGridEmergency(unsigned char eventDuration) = 0;
virtual std::future<ResponseCodes> basicLoadUp(unsigned char eventDuration) = 0;
virtual
                                std::future<ResponseCodes>
basicOutsideCommConnectionStatus(OutsideCommuncatonStatusCode code) = 0;

virtual      std::future<ResponseCodes>      basicPresentRelativePrice(unsigned      char
relativePriceIndicator) = 0;

virtual      std::future<ResponseCodes>      basicNextRelativePrice(unsigned      char
relativePriceIndicator) = 0;

virtual std::future<ResponseCodes> basicQueryOperationalState() = 0;
virtual std::future<ResponseCodes> basicShed(unsigned char eventDuration) = 0;
virtual std::future<ResponseCodes> basicPowerLevel(unsigned char powerLevel) = 0;

virtual std::future<ResponseCodes> intermediateSetAdvancedLoadUp(
    unsigned short duration,
    unsigned short value,
    unsigned char units,
    unsigned char suggestedEfficiency = 0xFF,

```

```
    unsigned int eventID = 0,

    unsigned int startTime = 0,

    unsigned char startRandomization = 0,

    unsigned char endRandomization = 0) = 0;

};

} /* namespace cea2045 */

#endif /* CEA2045_CEA2045_DEVICE_ICEA2045DEVICEUCM_H_ */
```

File: ../cea2045/cea2045/device/MessageHandler.cpp

```
/*

 * ResponseHandler.cpp

 *

 * Created on: Apr 5, 2016

 * Author: dupes

 */

#include "MessageHandler.h"

namespace cea2045

{

MessageHandler::MessageHandler()

{

}
```

```
MessageHandler::~MessageHandler()
```

```
{  
}
```

```
std::future<ResponseCodes> MessageHandler::queueRequest(std::unique_ptr<Message>  
request)
```

```
{
```

```
    std::lock_guard<std::mutex> lock(m_mutex);
```

```
    std::future<ResponseCodes> f = request->getFuture();
```

```
    m_requests.push_back(std::move(request));
```

```
    return f;
```

```
}
```

```
std::unique_ptr<Message> MessageHandler::getNextRequest()
```

```
{
```

```
    std::unique_ptr<Message> request;
```

```
    std::lock_guard<std::mutex> lock(m_mutex);
```

```
    if (m_requests.size() == 0)
```

```
        return nullptr;
```

```
request = std::move(m_requests.front());

m_requests.pop_front();

return request;
}

} /* namespace cea2045 */
```

File: ../cea2045/cea2045/device/MessageHandler.h

```
/*
 * ResponseHandler.h
 *
 * Created on: Apr 5, 2016
 * Author: dupes
 */

#ifndef CEA2045_CEA2045_LINKLAYER_REQUESTHANDLER_H_
#define CEA2045_CEA2045_LINKLAYER_REQUESTHANDLER_H_

#include "../linklayer/ILinkLayerCommSend.h"

#include <future>

#include <deque>

#include <memory>

#include <mutex>
```

```

#include "message/Message.h"

namespace cea2045

{

class MessageHandler

{

private:

    std::unique_ptr<Message> m_activeRequest;

    std::deque<std::unique_ptr<Message>> m_requests;

    std::mutex m_mutex;

public:

    MessageHandler();

    virtual ~MessageHandler();

    std::future<ResponseCodes> queueRequest(std::unique_ptr<Message> request);

    std::unique_ptr<Message> getNextRequest();

};

} /* namespace cea2045 */

#endif /* CEA2045_CEA2045_LINKLAYER_REQUESTHANDLER_H_ */

```

File: ../cea2045/cea2045/device/DeviceFactory.cpp

```

/*

 * DeviceFactory.cpp

```

```

*

*   Created on: Aug 25, 2015

*       Author: dupes

*/

#include "DeviceFactory.h"


#include "../linklayer/LinkLayerCommImpl.h"

#include "../processmessage/ProcessMessageUCM.h"

namespace cea2045 {

DeviceFactory::DeviceFactory()

{

}


DeviceFactory::~DeviceFactory()

{

}


ICEA2045DeviceUCM *DeviceFactory::createUCM(CommunicationPort *port, IUCM *ucm)

{

    std::unique_ptr<ProcessMessageUCM> processMessage =
std::unique_ptr<ProcessMessageUCM>(new ProcessMessageUCM(ucm));

    std::unique_ptr<LinkLayerCommImpl> linkLayerImpl =
std::unique_ptr<LinkLayerCommImpl>(new LinkLayerCommImpl(port));

```

```

    CEA2045DeviceUCM    *device    =    new    CEA2045DeviceUCM(std::move(linkLayerImpl),
std::move(processMessage));

    return device;

}

} /* namespace cea2045 */

```

File: ./cea2045/cea2045/device/ICEA2045Device.cpp

```

/*
 * ICEA2045Device.cpp
 *
 * Created on: Apr 26, 2016
 * Author: dupes
 */

```

```

#include "ICEA2045Device.h"

```

```

namespace cea2045

```

```

{

```

```

    ICEA2045Device::ICEA2045Device()

```

```

    {

```

```

    }

```



```
ICEA2045Device::~ICEA2045Device()
```

```
{  
  
}
```

```
} /* namespace cea2045 */
```

File: ./cea2045/cea2045/device/CEA2045Device.cpp

```
/*
```

```
 * CEA2045Device.cpp
```

```
 *
```

```
 * Created on: Mar 11, 2015
```

```
 * Author: dupes
```

```
*/
```

```
#include "CEA2045Device.h"
```

```
using namespace std;
```

```
namespace cea2045 {
```

```
CEA2045Device::CEA2045Device(ILinkLayerComm *linkLayer, IProcessMessage *processMessage,
```

```
unsigned int receiveTimeoutMS) :
```

```
    m_linkLayer(linkLayer),
```

```
    m_processMessage(processMessage),
```

```
    m_receiveTimeoutMS(receiveTimeoutMS),
```

```
    m_recvThread(NULL),
```



```
{  
  
    if (messageHeader->getLength() == 0)  
  
    {  
  
        m_processMessage->processMessageTypeSupported(m_linkLayer, messageHeader);  
  
    }  
  
    else if (messageHeader->isBasicMessage())  
  
    {  
  
        cea2045Basic *basic = (cea2045Basic *)messageHeader;  
  
  
        m_processMessage->processBasicMessage(m_linkLayer, basic);  
  
    }  
  
    else if (messageHeader->isDataLinkMessage())  
  
    {  
  
        m_processMessage->processDataLinkMessage(m_linkLayer, messageHeader);  
  
    }  
  
    else if (messageHeader->isIntermediateMessage())  
  
    {  
  
        m_processMessage->processIntermediateMessage(m_linkLayer, messageHeader);  
  
    }  
  
    else  
  
    {  
  
  
  
  
        m_processMessage->processInvalidMessage(m_linkLayer, messageHeader);  
  
    }  
  
}
```

```

void CEA2045Device::processResponse(cea2045MessageHeader *messageHeader, Message
*response)
{
    if (messageHeader->isBasicMessage())
    {
        cea2045Basic *basic = (cea2045Basic *)messageHeader;

        m_processMessage->processBasicMessage(m_linkLayer, basic);

        response->setResponseCode(ResponseCode::OK);
    }
    else if (messageHeader->isDataLinkMessage())
    {
        m_processMessage->processDataLinkMessage(m_linkLayer, messageHeader);

        response->setResponseCode(ResponseCode::OK);
    }
    else if (messageHeader->isIntermediateMessage())
    {
        m_processMessage->processIntermediateMessage(m_linkLayer, messageHeader);

        response->setResponseCode(ResponseCode::OK);
    }
    else
    {
        m_processMessage->processInvalidMessage(m_linkLayer, messageHeader);

        response->setResponseCode(ResponseCode::INVALID_RESPONSE);
    }
}

```

```

bool CEA2045Device::processReceiveBuffer(ReceiveBuffer &receiveBuffer)

{
    cea2045MessageHeader *messageHeader = (cea2045MessageHeader
*)receiveBuffer.getBuffer();

    if (receiveBuffer.getNumBytes() < 2)

        return false;

    if (receiveBuffer.getNumBytes() == 2 && messageHeader->isLinkLayerAckNak())
    {
        m_processMessage->processLinkLayerAckNak(m_linkLayer, messageHeader,
MessageCode::NONE);

        return true;
    }

    if (receiveBuffer.getNumBytes() <= sizeof(cea2045MessageHeader))

        return false;

    if (messageHeader->getLength() != receiveBuffer.getNumBytes() -
(sizeof(cea2045MessageHeader) + 2))
    {
        return false;
    }

```

```

if (!Checksum::validate(receiveBuffer.getBuffer(), receiveBuffer.getNumBytes()))
{
    m_linkLayer->sendLinkLayerNak(LinkLayerNakCode::CHECKSUM_ERROR);

    return true;
}

processRequest(messageHeader);

return true;
}

bool CEA2045Device::waitForResponse(Message *message)
{
    unsigned int messageTimeout = 3000;

    MTimer messageTimer(messageTimeout);

    ReceiveBuffer receiveBuffer(4096);

    cea2045MessageHeader *messageHeader = (cea2045MessageHeader
*)receiveBuffer.getBuffer();

    while (true)
    {
        unsigned long int elapsed = messageTimer.getElapsedMS();

        if (messageTimer.isExpired() || m_linkLayer->recv(receiveBuffer, messageTimeout -

```

```

elapsed) == 0)

{

    if (receiveBuffer.getNumBytes() > 0)

    {

        m_processMessage->processIncompleteMessage(m_linkLayer, receiveBuffer.getBuffer(),

            receiveBuffer.getNumBytes());

    }


    message->setResponseCode(ResponseCode::TIMEOUT);


    return false;
}


if (receiveBuffer.getNumBytes() >= sizeof(cea2045MessageHeader))

{

    if (messageHeader->getLength() ==

        receiveBuffer.getNumBytes() - (sizeof(cea2045MessageHeader) + 2))

    {

        if (!Checksum::validate(receiveBuffer.getBuffer(), receiveBuffer.getNumBytes()))

        {

            m_linkLayer->sendLinkLayerNak(LinkLayerNakCode::CHECKSUM_ERROR);

            message->setResponseCode(ResponseCode::BAD_CRC);


            return false;

        }

    }

    else

```

```

{
    processResponse(messageHeader, message);

    return true;
}

}

}

}

}

bool CEA2045Device::waitForAck(Message *message)
{
    unsigned int ackTimeout = 200;

    MTimer ackTimer(ackTimeout);

    ReceiveBuffer receiveBuffer(4096);

    while (true)
    {
        unsigned long int elapsed = ackTimer.getElapsedMS();

        if (ackTimer.isExpired())
        {
            message->setResponseCode(ResponseCode::NO_ACK_RECEIVED);

            return false;
        }
    }
}

```



```
}
```

```
if (m_linkLayer->recv(receiveBuffer, ackTimeout - elapsed) == 0)
```

```
{
```

```
    message->setResponseCode(ResponseCode::NO_ACK_RECEIVED);
```

```
    return false;
```

```
}
```

```
if (receiveBuffer.getNumBytes() > 2)
```

```
{
```

```
    message->setResponseCode(ResponseCode::NO_ACK_RECEIVED);
```

```
    return false;
```

```
}
```

```
if (receiveBuffer.getNumBytes() == 2)
```

```
{
```

```
    break;
```

```
}
```

```
}
```

```
cea2045MessageHeader *messageHeader = (cea2045MessageHeader
```

```
*)receiveBuffer.getBuffer();
```

```
if (!messageHeader->isLinkLayerAckNak())
```

```
{
```

```
    message->setResponseCode(ResponseCode::NO_ACK_RECEIVED);
```

```

        return false;
    }

    m_processMessage->processLinkLayerAckNak(m_linkLayer,                messageHeader,
message->getMessageCode());

    if (messageHeader->isLinkLayerNak())
    {
        message->setResponseCodeNak(ConvertEnums::convertLinkLayerNak(messageHeader->msgType2)
);
        return false;
    }

    return true;
}

```

```

void CEA2045Device::processNextRequest()
{
    auto request = m_requestHandler.getNextRequest();

    if (request == nullptr)
        return;

    m_linkLayer->sendRequest(request->getBuffer(),
        request->getNumBytes());
}

```

```

if (!waitForAck(request.get()))

    return;

if (request->getMessageCode() == MessageCode::SUPPORT_DATA_LINK_MESSAGES ||

    request->getMessageCode() == MessageCode::SUPPORT_INTERMEDIATE_MESSAGES)

{

    request->setResponseCode(ResponseCode::OK);

    return;

}

waitForResponse(request.get());

}

```

```

void CEA2045Device::recvOne(ReceiveBuffer &receiveBuffer)

{

    int currentBytes = receiveBuffer.getNumBytes();

    int bytesRead = m_linkLayer->recv(receiveBuffer, 100);

    if (currentBytes == 0 && bytesRead > 0)

    {

        m_receiveTimer.reset();

    }

    if (bytesRead > 0)

    {

        if (processReceiveBuffer(receiveBuffer))

```

```

{
    receiveBuffer.reset();
}

}

if (receiveBuffer.getNumBytes() > 0 && m_receiveTimer.getElapsedMS() >=
m_receiveTimeoutMS)
{
    m_processMessage->processIncompleteMessage(m_linkLayer, receiveBuffer.getBuffer(),
receiveBuffer.getNumBytes());

    receiveBuffer.reset();
}

if (currentBytes == 0 && bytesRead == 0)
{
    processNextRequest();

    m_requestTimer.reset();
}
}

void CEA2045Device::recv()
{
    ReceiveBuffer receiveBuffer(4096);

    while (!shuttingDown())

```

```
{  
    recvOne(receiveBuffer);  
}  
}
```

```
bool CEA2045Device::start()
```

```
{  
    if (m_isRunning)  
        return false;  
  
    m_isRunning = true;  
  
    m_shutdown = false;  
  
    m_recvThread = new thread(&CEA2045Device::recv, this);  
  
    return true;  
}
```

```
void CEA2045Device::shutDown()
```

```
{  
    if (m_recvThread != NULL)  
    {  
        m_shutdown = true;  
    }
```

```

    m_recvThread->join();

    delete (m_recvThread);

    m_isRunning = false;

    m_recvThread = NULL;
}

}

bool CEA2045Device::shuttingDown()
{
    return m_shutdown;
}

std::future<ResponseCodes> CEA2045Device::queueRequest(Message *request)
{
    return m_requestHandler.queueRequest(std::unique_ptr<Message>(request));
}

std::future<ResponseCodes> CEA2045Device::queryMaxPayload()
{
    return queueRequest(new QueryMaxPayload());
}

```

```

std::future<ResponseCodes> CEA2045Device::queryMessageTypeSupported(MessageCode
messageCode, unsigned char messageType1, unsigned char messageType2)
{
    return queueRequest(new QueryMessageTypeSupported(messageCode, messageType1,
messageType2));
}

```

```

std::future<ResponseCodes> CEA2045Device::querySupportDataLinkMessages()
{
    return queryMessageTypeSupported(MessageCode::SUPPORT_DATA_LINK_MESSAGES,
DATA_LINK_MSG_TYP1, DATA_LINK_MSG_TYP2);
}

```

```

std::future<ResponseCodes> CEA2045Device::querySupportIntermediateMessages()
{
    return queryMessageTypeSupported(MessageCode::SUPPORT_INTERMEDIATE_MESSAGES,
INTERMEDIATE_MSG_TYP1, INTERMEDIATE_MSG_TYP2);
}

```

```

} // namespace cea2045

```

File: ./cea2045/cea2045/device/CEA2045Device.h

```

/*
 * CEA2045Device.h

```

```
*

*   Created on: Mar 11, 2015

*       Author: dupes

*/


#ifndef CEA2045DEVICE_H_
#define CEA2045DEVICE_H_


#include <memory>

#include <future>

#include <thread>

#include <time.h>

#include "../util/MSTimer.h"

#include "../util/Checksum.h"

#include "../linklayer/ILinkLayerComm.h"

#include "../message/CEA2045Message.h"

#include "../processmessage/IProcessMessage.h"

#include "message/QueryMaxPayload.h"

#include "message/QueryMessageTypeSupported.h"

#include "MessageHandler.h"


namespace cea2045 {

class CEA2045Device

{

private:
```



```
std::unique_ptr<ILinkLayerComm> m_p_linkLayer;

std::unique_ptr<IProcessMessage> m_p_processMessage;


ILinkLayerComm *m_linkLayer;

IProcessMessage *m_processMessage;


unsigned int m_receiveTimeoutMS;


std::thread *m_recvThread;


MessageHandler m_requestHandler;


bool m_shutdown;

bool m_isRunning;


MSTimer m_receiveTimer;

MSTimer m_requestTimer;


void processRequest(cea2045MessageHeader *messageHeader);

void processResponse(cea2045MessageHeader *messageHeader, Message *response);

bool processReceiveBuffer(ReceiveBuffer &receiveBuffer);


bool waitForResponse(Message *message);

bool waitForAck(Message *message);


void recv();
```

protected:

public:

```
CEA2045Device(ILinkLayerComm *linkLayer, IProcessMessage *processMessage, unsigned int  
receiveTimeoutMS = 500);
```

```
CEA2045Device(std::unique_ptr<ILinkLayerComm> linkLayer,  
std::unique_ptr<IProcessMessage> processMessage, unsigned int receiveTimeoutMS = 500);
```

```
virtual ~CEA2045Device();
```

```
std::future<ResponseCodes> queueRequest(Message *request);
```

```
std::future<ResponseCodes> queryMaxPayload();
```

```
std::future<ResponseCodes> queryMessageTypeSupported(MessageCode messageCode, unsigned  
char messageType1, unsigned char messageType2);
```

```
std::future<ResponseCodes> querySupportDataLinkMessages();
```

```
std::future<ResponseCodes> querySupportIntermediateMessages();
```

```
bool start();
```

```
void shutDown();
```

```
bool shuttingDown();
```

```
void recvOne(ReceiveBuffer &receiveBuffer);

void processNextRequest();

};

} // namespace cea2045

#endif /* CEA2045DEVICE_H_ */
```

File: ./cea2045/cea2045/device/ICEA2045Device.h

```
/*
 * ICEA2045Device.h
 *
 * Created on: Apr 26, 2016
 * Author: dupes
 */

#ifndef CEA2045_CEA2045_DEVICE_ICEA2045DEVICE_H_
#define CEA2045_CEA2045_DEVICE_ICEA2045DEVICE_H_

namespace cea2045

{

class ICEA2045Device
{
public:

    ICEA2045Device();
```

```

    virtual ~ICEA2045Device();

};

} /* namespace cea2045 */

#endif /* CEA2045_CEA2045_DEVICE_ICEA2045DEVICE_H_ */

```

File: ./cea2045/cea2045/device/CEA2045DeviceUCM.cpp

```

/*
 * CEA2045DeviceUCM.cpp
 *
 * Created on: Aug 25, 2015
 * Author: dupes
 */

#include "CEA2045DeviceUCM.h"

#include "message/SetAdvancedLoadUp.h"

namespace cea2045 {

CEA2045DeviceUCM::CEA2045DeviceUCM(std::unique_ptr<ILinkLayerComm> linkLayer,
std::unique_ptr<IProcessMessage> processMessage) :

    CEA2045Device(std::move(linkLayer), std::move(processMessage))

{

}

CEA2045DeviceUCM::CEA2045DeviceUCM(ILinkLayerComm *linkLayer, IProcessMessage

```

```
*processMessage) :
```

```
    CEA2045Device(linkLayer, processMessage)
```

```
{
```

```
}
```

```
CEA2045DeviceUCM::~~CEA2045DeviceUCM()
```

```
{
```

```
}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::intermediateGetDeviceInformation()
```

```
{
```

```
    return queueRequest(new Intermediate(MessageCode::DEVICE_INFORMATION_REQUEST,  
        INFO_REQ, INFO_REQ));
```

```
}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::intermediateGetCommodity()
```

```
{
```

```
    return queueRequest(new Intermediate(MessageCode::GET_COMMODITY_REQUEST,  
        COMMODITY_READ, CLEAR_OP_CODE2));
```

```
}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::intermediateGetTemperatureOffset()
```

```
{
```

```
    return queueRequest(new Intermediate(MessageCode::GET_TEMPERATURE_OFFSET,
```

```
    GET_SET, TEMP_OFFSET));  
  
}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::intermediateGetSetPoint()  
{  
    return queueRequest(new Intermediate(MessageCode::GET_SETPPOINTS_REQUEST,  
        GET_SET, TEMP_SETPPOINT));  
}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::intermediateGetPresentTemperature()  
{  
    return queueRequest(new Intermediate(MessageCode::GET_PRESENT_TEMPERATURE_REQUEST,  
        GET_SET, PRESENT_TEMPERATURE));  
}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::intermediateSetTemperatureOffset(unsigned  
char temperatureOffset, TemperatureUnits units)  
{  
    return queueRequest(new SetTemperatureOffset(temperatureOffset, units));  
}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::intermediateSetAdvancedLoadUp(  
    unsigned short duration,
```

```

    unsigned short value,

    unsigned char units,

    unsigned char suggestedEfficiency,

    unsigned int eventID,

    unsigned int startTime,

    unsigned char startRandomization,

    unsigned char endRandomization) {

    auto setAdvancedLoadUp = new SetAdvancedLoadUp(

        duration,

        value,

        units,

        suggestedEfficiency,

        eventID,

        startTime,

        startRandomization,

        endRandomization

    );

    return queueRequest(setAdvancedLoadUp);

}

std::future<ResponseCodes>    CEA2045DeviceUCM::intermediateSetSetpoints(unsigned    short
deviceType, TemperatureUnits units, unsigned short setpoint1, unsigned short setpoint2)
{

    return queueRequest(new SetSetpoints(deviceType, units, setpoint1, setpoint2));
}

```

```
}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::intermediateSetEnergyPrice(unsigned short
currentPrice, unsigned short currencyCode,
    unsigned char digitsAfterDecimalPoint, unsigned int expirationDatetimeUTC,
    unsigned short nextPrice)
{
    return queueRequest(new SetEnergyPrice(currentPrice, currencyCode,
digitsAfterDecimalPoint, expirationDatetimeUTC,
    nextPrice));
}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::intermediateStartCycling(unsigned int
eventID, unsigned int startTimeUTCSince2000, unsigned short durationInMinutes,
    unsigned char dutyCycle, unsigned char startRandomizationMinutes, unsigned char
endRandomizationMintues,
    unsigned char criticality)
{
    return queueRequest(new StartCycling(eventID, startTimeUTCSince2000, durationInMinutes,
    dutyCycle, startRandomizationMinutes, endRandomizationMintues, criticality));
}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::intermediateTerminateCycling(unsigned int
eventID, unsigned char endRandomizationInMinutes)
```



```
{  
    return queueRequest(new TerminateCycling(eventID, endRandomizationInMinutes));  
}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::basicCriticalPeakEvent(unsigned char  
eventDuration)
```

```
{  
    return queueRequest(new Basic(MessageCode::BASIC_CRITICAL_PEAK_EVENT_REQUEST,  
        CPP, eventDuration));  
}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::basicEndShed(unsigned char  
durationToNextEvent)
```

```
{  
    return queueRequest(new Basic(MessageCode::BASIC_END_SHED_REQUEST,  
        END_SHED, durationToNextEvent));  
}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::basicGridEmergency(unsigned char  
eventDuration)
```

```
{  
    return queueRequest(new Basic(MessageCode::BASIC_GRID_EMERGENCY_REQUEST,  
        GRID_EMERGENCY, eventDuration));  
}
```

```

std::future<ResponseCodes> CEA2045DeviceUCM::basicLoadUp(unsigned char eventDuration)
{
    return queueRequest(new Basic(MessageCode::BASIC_LOAD_UP_REQUEST,
        LOADUP, eventDuration));
}

```

```

std::future<ResponseCodes>
CEA2045DeviceUCM::basicOutsideCommConnectionStatus(OutsideCommuncatonStatusCode code)
{
    return queueRequest(new
        Basic(MessageCode::BASIC_OUTSIDE_COMM_CONNECTION_STATUS_MESSAGE,
            COMM_STATUS, (unsigned char)code));
}

```

```

std::future<ResponseCodes> CEA2045DeviceUCM::basicPresentRelativePrice(unsigned char
relativePriceIndicator)
{
    return queueRequest(new Basic(MessageCode::BASIC_PRESENT_RELATIVE_PRICE_REQUEST,
        PRESENT_RELATIVE_PRICE, relativePriceIndicator));
}

```

```

std::future<ResponseCodes> CEA2045DeviceUCM::basicNextRelativePrice(unsigned char

```

```
relativePriceIndicator)

{

    return queueRequest(new Basic(MessageCode::BASIC_NEXT_RELATIVE_PRICE_REQUEST,

        NEXT_RELATIVE_PRICE, relativePriceIndicator));

}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::basicQueryOperationalState()

{

    return queueRequest(new Basic(MessageCode::BASIC_QUERY_OPERATIONAL_STATE_REQUEST,

        OPER_STATE_REQ, CLEAR_OP_CODE2));

}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::basicShed(unsigned char eventDuration)

{

    return queueRequest(new Basic(MessageCode::BASIC_SHED_REQUEST,

        SHED, eventDuration));

}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::basicPowerLevel(unsigned char powerLevel)

{

    return queueRequest(new Basic(MessageCode::BASIC_POWER_LEVEL,

        POWER_LEVEL, powerLevel));

}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::queryMaxPayload()
```

```
{  
    return CEA2045Device::queryMaxPayload();  
}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::queryMessageTypeSupported(MessageCode  
messageCode, unsigned char messageType1, unsigned char messageType2)
```

```
{  
    return CEA2045Device::queryMessageTypeSupported(messageCode, messageType1,  
messageType2);  
}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::querySupportDataLinkMessages()
```

```
{  
    return CEA2045Device::querySupportDataLinkMessages();  
}
```

```
std::future<ResponseCodes> CEA2045DeviceUCM::querySupportIntermediateMessages()
```

```
{  
    return CEA2045Device::querySupportIntermediateMessages();  
}
```

```
bool CEA2045DeviceUCM::start()

{

    return CEA2045Device::start();

}
```

```
void CEA2045DeviceUCM::shutDown()

{

    return CEA2045Device::shutDown();

}
```

```
} /* namespace cea2045 */
```

File: ../cea2045/cea2045/device/message/TerminateCycling.cpp

```
/*

* TerminateCycling.cpp

*

* Created on: Apr 8, 2016

* Author: dupes

*/

#include "../message/TerminateCycling.h"

namespace cea2045

{
```

```
TerminateCycling::TerminateCycling(unsigned int eventID, unsigned char
```

```
endRandomizationInMinutes) :
```

```
    Message(MessageCode::TERMINATE_CYCLING_REQUEST)
```

```
{  
  
    m_msg.msgType1 = INTERMEDIATE_MSG_TYP1;  
  
    m_msg.msgType2 = INTERMEDIATE_MSG_TYP2;  
  
    m_msg.setLength();  
  
    m_msg.opCode1 = AUTO_CYCLE;  
  
    m_msg.opCode2 = AC_STOP;  
  
    m_msg.setEventID(eventID);  
  
    m_msg.endRandomizationInMinutes = endRandomizationInMinutes;  
  
  
    m_msg.setChecksum();  
  
}
```

```
TerminateCycling::~~TerminateCycling()
```

```
{  
  
}
```

```
int TerminateCycling::getNumBytes()
```

```
{  
  
    return sizeof(m_msg);  
  
}
```

```
unsigned char* TerminateCycling::getBuffer()
```

```
{  
  
    return (unsigned char *)&m_msg;  
  
}
```

```
} /* namespace cea2045 */
```

File: ./cea2045/cea2045/device/message/SetEnergyPrice.h

```
/*  
  
 * SetEnergyPrice.h  
  
 *  
  
 * Created on: Apr 7, 2016  
  
 * Author: dupes  
  
 */  
  
#ifndef CEA2045_CEA2045_DEVICE_MESSAGE_SETENERGYPRICE_H_  
#define CEA2045_CEA2045_DEVICE_MESSAGE_SETENERGYPRICE_H_  
  
#include "../..message/CEA2045MessageSetEnergyPriceRequest.h"  
#include "../message/Message.h"  
  
namespace cea2045  
{  
  
class SetEnergyPrice: public Message  
{  
  
private:  
  
    cea2045SetEnergyPriceRequest m_msg;
```

```

public:

    SetEnergyPrice(unsigned short currentPrice, unsigned short currencyCode,
        unsigned char digitsAfterDecimalPoint, unsigned int expirationDatetimeUTC,
        unsigned short nextPrice);

    virtual ~SetEnergyPrice();

    virtual int getNumBytes();

    virtual unsigned char *getBuffer();

};

} /* namespace cea2045 */

#endif /* CEA2045_CEA2045_DEVICE_MESSAGE_SETENERGYPRICE_H_ */

```

File: ./cea2045/cea2045/device/message/Message.h

```

/*
 * Request.h
 *
 * Created on: Apr 5, 2016
 * Author: dupes
 */

#ifndef CEA2045_CEA2045_DEVICE_MESSAGE_MESSAGE_H_
#define CEA2045_CEA2045_DEVICE_MESSAGE_MESSAGE_H_

```



```
#include "../util/Checksum.h"
```

```
#include "../message/ConvertEnums.h"
```

```
#include <future>
```

```
namespace cea2045
```

```
{
```

```
enum class ResponseCode
```

```
{
```

```
    OK,
```

```
    TIMEOUT,
```

```
    BAD_CRC,
```

```
    INVALID_RESPONSE,
```

```
    NO_ACK_RECEIVED,
```

```
    NAK
```

```
};
```

```
struct ResponseCodes
```

```
{
```

```
    ResponseCode responsesCode;
```

```
    LinkLayerNakCode nak;
```

```
};
```

```
class Message
```

```
{
```

```
private:
```

```

std::promise<ResponseCodes> m_promise;

MessageCode m_messageCode;

public:

    Message(MessageCode messageCode);

    virtual ~Message();

    virtual int getNumBytes() = 0;

    virtual unsigned char *getBuffer() = 0;

    void setResponseCode(ResponseCode responseCode);

    void setResponseCodeNak(LinkLayerNakCode nak);

    std::future<ResponseCodes> getFuture();

    MessageCode getMessageCode();

};

} /* namespace cea2045 */

#endif /* CEA2045_CEA2045_DEVICE_MESSAGE_MESSAGE_H_ */

```

File: ../cea2045/cea2045/device/message/SetEnergyPrice.cpp

```

/*
 * SetEnergyPrice.cpp
 *
 * Created on: Apr 7, 2016

```

```

*      Author: dupes

*/

#include "../message/SetEnergyPrice.h"

namespace cea2045

{

SetEnergyPrice::SetEnergyPrice(unsigned short currentPrice, unsigned short currencyCode,
    unsigned char digitsAfterDecimalPoint, unsigned int expirationDatetimeUTC,
    unsigned short nextPrice) :
    Message(MessageCode::SET_ENERGY_PRICE_REQUEST)
{

    m_msg.msgType1 = INTERMEDIATE_MSG_TYP1;

    m_msg.msgType2 = INTERMEDIATE_MSG_TYP2;

    m_msg.setLength();

    m_msg.opCode1 = GET_SET;

    m_msg.opCode2 = ENERGY_PRICE;

    m_msg.setCurrentPrice(currentPrice);

    m_msg.setCurrencyCode(currencyCode);

    m_msg.digitsAfterDecimalPoint = digitsAfterDecimalPoint;

    m_msg.setExpirationDatetimeUTC(expirationDatetimeUTC);

    m_msg.setNextPrice(nextPrice);

    m_msg.setChecksum();

}

```

```
SetEnergyPrice::~SetEnergyPrice()
```

```
{  
}
```

```
int SetEnergyPrice::getNumBytes()
```

```
{  
    return sizeof(m_msg);  
}
```

```
unsigned char* SetEnergyPrice::getBuffer()
```

```
{  
    return (unsigned char *)&m_msg;  
}
```

```
} /* namespace cea2045 */
```

File: ../cea2045/cea2045/device/message/TerminateCycling.h

```
/*  
 * TerminateCycling.h  
 *  
 * Created on: Apr 8, 2016  
 * Author: dupes  
 */
```

```

#ifndef CEA2045_CEA2045_DEVICE_MESSAGE_TERMINATECYCLING_H_

#define CEA2045_CEA2045_DEVICE_MESSAGE_TERMINATECYCLING_H_

#include "../message/CEA2045MessageTerminateCyclingRequest.h"

#include "../message/Message.h"

namespace cea2045

{

class TerminateCycling: public Message

{

private:

    cea2045TerminateCyclingRequest m_msg;

public:

    TerminateCycling(unsigned int eventID, unsigned char endRandomizationInMinutes);

    virtual ~TerminateCycling();

    virtual int getNumBytes();

    virtual unsigned char *getBuffer();

};

} /* namespace cea2045 */

#endif /* CEA2045_CEA2045_DEVICE_MESSAGE_TERMINATECYCLING_H_ */

```

File: ../cea2045/cea2045/device/message/StartCycling.cpp

```

/*
 * StartCycling.cpp
 *
 * Created on: Apr 7, 2016
 * Author: dupes
 */

#include "../message/StartCycling.h"

namespace cea2045
{

StartCycling::StartCycling(unsigned int eventID, unsigned int startTimeUTCSince2000,
unsigned short durationInMinutes,
    unsigned char dutyCycle, unsigned char startRandomizationMinutes, unsigned char
endRandomizationMintues,
    unsigned char criticality) :
    Message(MessageCode::START_CYCLING_REQUEST)
{

    m_msg.msgType1 = INTERMEDIATE_MSG_TYP1;

    m_msg.msgType2 = INTERMEDIATE_MSG_TYP2;

    m_msg.setLength();

    m_msg.opCode1 = AUTO_CYCLE;

    m_msg.opCode2 = AC_START;

    m_msg.setEventID(eventID);

    m_msg.setStartTimeUTCSince2000(startTimeUTCSince2000);

    m_msg.setDurationInMinutes(durationInMinutes);

```

```
m_msg.dutyCycle = dutyCycle;

m_msg.startRandomizationInMinutes = startRandomizationMinutes;

m_msg.endRandomizationInMinutes = endRandomizationMintues;

m_msg.criticality = criticality;


m_msg.setChecksum();

}
```

```
StartCycling::~StartCycling()
```

```
{

}
```

```
int StartCycling::getNumBytes()
```

```
{

    return sizeof(m_msg);

}
```

```
unsigned char* StartCycling::getBuffer()
```

```
{

    return (unsigned char *)&m_msg;

}
```

```
} /* namespace cea2045 */
```

File: ./cea2045/cea2045/device/message/StartCycling.h

```
/*
 * StartCycling.h
 *
 * Created on: Apr 7, 2016
 * Author: dupes
 */

#ifndef CEA2045_CEA2045_DEVICE_MESSAGE_STARTCYCLING_H_
#define CEA2045_CEA2045_DEVICE_MESSAGE_STARTCYCLING_H_

#include "../message/CEA2045MessageStartCyclingRequest.h"
#include "../message/Message.h"

namespace cea2045
{

class StartCycling : public Message
{
private:
    cea2045StartCyclingRequest m_msg;

public:
    StartCycling(unsigned int eventID, unsigned int startTimeUTCSince2000, unsigned short
durationInMinutes,
        unsigned char dutyCycle, unsigned char startRandomizationMinutes, unsigned char
endRandomizationMintues,
```



```

        unsigned char criticality);

virtual ~StartCycling();

virtual int getNumBytes();

virtual unsigned char *getBuffer();

};

} /* namespace cea2045 */

#endif /* CEA2045_CEA2045_DEVICE_MESSAGE_STARTCYCLING_H_ */

```

File: ../cea2045/cea2045/device/message/Intermediate.cpp

```

/*
 * Intermediate.cpp
 *
 * Created on: Apr 6, 2016
 * Author: dupes
 */

#include "../message/Intermediate.h"

namespace cea2045
{

Intermediate::Intermediate(MessageCode messageCode, unsigned char opCode1, unsigned char
opCode2) :

```

```
Message(messageCode)

{

    m_msg.msgType1 = INTERMEDIATE_MSG_TYP1;

    m_msg.msgType2 = INTERMEDIATE_MSG_TYP2;

    m_msg.setLength();

    m_msg.opCode1 = opCode1;

    m_msg.opCode2 = opCode2;


    m_msg.setChecksum();

}
```

```
Intermediate::~Intermediate()
```

```
{

}
```

```
int Intermediate::getNumBytes()
```

```
{

    return sizeof(m_msg);

}
```

```
unsigned char *Intermediate::getBuffer()
```

```
{

    return (unsigned char *)&m_msg;

}
```

```
} /* namespace cea2045 */
```

File: ./cea2045/cea2045/device/message/SetAdvancedLoadUp.h

```
#ifndef CEA2045_SETADVANCEDLOADUP_H_
```

```
#define CEA2045_SETADVANCEDLOADUP_H_
```

```
#include "Message.h"
```

```
#include "../../util/Checksum.h"
```

```
#include "../../message/CEA2045Message.h" // Add this include for cea2045MessageHeader
```

```
namespace cea2045 {
```

```
class SetAdvancedLoadUp : public Message {
```

```
private:
```

```
    struct SetAdvancedLoadUpMsg {
```

```
        struct cea2045MessageHeader header; // Use struct keyword here
```

```
        unsigned char opCode1;
```

```
        unsigned char opCode2;
```

```
        unsigned short duration;
```

```
        unsigned short value;
```

```
        unsigned char units;
```

```
        unsigned char suggestedEfficiency;
```

```
        unsigned int eventID;
```

```
        unsigned int startTime;
```

```
        unsigned char startRandomization;
```

```
        unsigned char endRandomization;
```

```

    unsigned short checksum;

    void setLength() {
        header.length = htons(sizeof(SetAdvancedLoadUpMsg) -
sizeof(cea2045MessageHeader));
    }

    void setChecksum() {
        this->checksum = Checksum::calculate((unsigned char*)this,
sizeof(SetAdvancedLoadUpMsg) - sizeof(unsigned short));
    }
} __attribute__((packed));

SetAdvancedLoadUpMsg m_msg;

public:
    SetAdvancedLoadUp(unsigned short duration, unsigned short value, unsigned char
units,
        unsigned char suggestedEfficiency = 0xFF, unsigned int eventID = 0,
        unsigned int startTime = 0, unsigned char startRandomization = 0,
        unsigned char endRandomization = 0);

    virtual ~SetAdvancedLoadUp();

    virtual int getNumBytes();

    virtual unsigned char *getBuffer();
};

```

```
} /* namespace cea2045 */
```

```
#endif /* CEA2045_SETADVANCEDLOADUP_H_ */
```

File: ./cea2045/cea2045/device/message/QueryMessageTypeSupported.cpp

```
/*
```

```
 * QueryMessageTypeSupported.cpp
```

```
 *
```

```
 * Created on: Apr 6, 2016
```

```
 * Author: dupes
```

```
 */
```

```
#include "../message/QueryMessageTypeSupported.h"
```

```
namespace cea2045
```

```
{
```

```
QueryMessageTypeSupported::QueryMessageTypeSupported(MessageCode messageCode, unsigned  
char messageType1, unsigned char messageType2) :
```

```
    Message(messageCode)
```

```
{
```

```
    m_msg.msgType1 = messageType1;
```

```
    m_msg.msgType2 = messageType2;
```

```
    m_msg.length = 0x00;
```

```
    m_msg.setChecksum();
```

```
}
```

```
QueryMessageTypeSupported::~QueryMessageTypeSupported()
```

```
{
```

```
}
```

```
int QueryMessageTypeSupported::getNumBytes()
```

```
{
```

```
    return sizeof(m_msg);
```

```
}
```

```
unsigned char* QueryMessageTypeSupported::getBuffer()
```

```
{
```

```
    return (unsigned char *)&m_msg;
```

```
}
```

```
} /* namespace cea2045 */
```

File: ./cea2045/cea2045/device/message/QueryMaxPayload.h

```
/*
```

```
 * QueryMessageTypeSupported.h
```

```
 *
```

```
 * Created on: Apr 5, 2016
```

```
 * Author: dupes
```

```
*/

#ifndef CEA2045_CEA2045_DEVICE_MESSAGE_QUERYMAXPAYLOAD_H_
#define CEA2045_CEA2045_DEVICE_MESSAGE_QUERYMAXPAYLOAD_H_

#include "../message/CEA2045MessageDataLink.h"
#include "../message/Message.h"

namespace cea2045

{

class QueryMaxPayload : public Message
{
private:
    cea2045DataLink m_msg;

public:
    QueryMaxPayload();
    virtual ~QueryMaxPayload();

    virtual int getNumBytes();
    virtual unsigned char *getBuffer();
};

} /* namespace cea2045 */

#endif /* CEA2045_CEA2045_DEVICE_MESSAGE_QUERYMAXPAYLOAD_H_ */
```

File: ./cea2045/cea2045/device/message/Intermediate.h

```
/*
 * Intermediate.h
 *
 * Created on: Apr 6, 2016
 * Author: dupes
 */

#ifndef CEA2045_CEA2045_DEVICE_MESSAGE_INTERMEDIATE_H_
#define CEA2045_CEA2045_DEVICE_MESSAGE_INTERMEDIATE_H_

#include "../message/CEA2045MessageIntermediate.h"
#include "../message/Message.h"

namespace cea2045
{
    class Intermediate : public Message
    {
    private:
        cea2045Intermediate m_msg;

    public:
        Intermediate(MessageCode messageCode, unsigned char opCode1, unsigned char opCode2);
        virtual ~Intermediate();

        virtual int getNumBytes();
    };
}
```



```
virtual unsigned char *getBuffer();
```

```
};
```

```
} /* namespace cea2045 */
```

```
#endif /* CEA2045_CEA2045_DEVICE_MESSAGE_INTERMEDIATE_H_ */
```

File: ../cea2045/cea2045/device/message/QueryMaxPayload.cpp

```
/*
```

```
 * QueryMessageTypeSupported.cpp
```

```
 *
```

```
 * Created on: Apr 5, 2016
```

```
 * Author: dupes
```

```
 */
```

```
#include "../message/QueryMaxPayload.h"
```

```
namespace cea2045
```

```
{
```

```
QueryMaxPayload::QueryMaxPayload() :
```

```
    Message(MessageCode::MAX_PAYLOAD_REQUEST)
```

```
{
```

```
    m_msg.msgType1 = DATALINK_MSG_TYP1;
```

```
    m_msg.msgType2 = DATALINK_MSG_TYP2;
```

```
    m_msg.setLength();
```

```
    m_msg.opCode1 = MAXPAYLOAD_REQ_OP_CODE1;
```

```
m_msg.opCode2 = CLEAR_OP_CODE2;
```

```
m_msg.setChecksum();
```

```
}
```

```
QueryMaxPayload::~QueryMaxPayload()
```

```
{
```

```
}
```

```
int QueryMaxPayload::getNumBytes()
```

```
{
```

```
    return sizeof(m_msg);
```

```
}
```

```
unsigned char *QueryMaxPayload::getBuffer()
```

```
{
```

```
    return (unsigned char *)&m_msg;
```

```
}
```

```
} /* namespace cea2045 */
```

File: ./cea2045/cea2045/device/message/Basic.cpp

```
/*
```

```
 * Basic.cpp
```

```
*  
  
*   Created on: Apr 6, 2016  
  
*       Author: dupes  
  
*/
```

```
#include "../message/Basic.h"
```

```
namespace cea2045
```

```
{
```

```
Basic::Basic(MessageCode messageCode, unsigned char opCode1, unsigned char opCode2) :
```

```
    Message(messageCode)
```

```
{
```

```
    m_msg.msgType1 = BASIC_MSG_TYP1;
```

```
    m_msg.msgType2 = BASIC_MSG_TYP2;
```

```
    m_msg.setLength();
```

```
    m_msg.opCode1 = opCode1;
```

```
    m_msg.opCode2 = opCode2;
```

```
    m_msg.setChecksum();
```

```
}
```

```
Basic::~~Basic()
```

```
{
```

```
}
```

```
int Basic::getNumBytes()

{

    return sizeof(m_msg);

}


unsigned char* Basic::getBuffer()

{

    return (unsigned char *)&m_msg;

}


} /* namespace cea2045 */
```

File: ./cea2045/cea2045/device/message/Basic.h

```
/*

 * Basic.h

 *

 * Created on: Apr 6, 2016

 * Author: dupes

 */


#ifndef CEA2045_CEA2045_DEVICE_MESSAGE_BASIC_H_

#define CEA2045_CEA2045_DEVICE_MESSAGE_BASIC_H_


#include "../message/CEA2045MessageBasic.h"

#include "../message/Message.h"
```

```

namespace cea2045

{

class Basic : public Message
{
private:

    cea2045Basic m_msg;

public:

    Basic(MessageCode messageCode, unsigned char opCode1, unsigned char opCode2);

    virtual ~Basic();

    virtual int getNumBytes();

    virtual unsigned char *getBuffer();

};

} /* namespace cea2045 */

#endif /* CEA2045_CEA2045_DEVICE_MESSAGE_BASIC_H_ */

```

File: ./cea2045/cea2045/device/message/Message.cpp

```

/*

* Request.cpp

*

* Created on: Apr 5, 2016

* Author: dupes

```

```
*/
```

```
#include "../message/Message.h"
```

```
namespace cea2045
```

```
{
```

```
Message::Message(MessageCode messageCode) :
```

```
    m_messageCode(messageCode)
```

```
{
```

```
}
```

```
Message::~Message()
```

```
{
```

```
}
```

```
void Message::setResponseCode(ResponseCode responseCode)
```

```
{
```

```
    ResponseCodes responsesCodes;
```

```
    responsesCodes.responsesCode = responseCode;
```

```
    responsesCodes.nak = LinkLayerNakCode::NONE;
```

```
    m_promise.set_value(responsesCodes);
```

```
}
```

```
void Message::setResponseCodeNak(LinkLayerNakCode nak)
```

```
{
```

```
    ResponseCodes responsesCodes;
```

```
    responsesCodes.responsesCode = ResponseCode::NAK;
```

```
    responsesCodes.nak = nak;
```

```
    m_promise.set_value(responsesCodes);
```

```
}
```

```
std::future<ResponseCodes> Message::getFuture()
```

```
{
```

```
    return m_promise.get_future();
```

```
}
```

```
MessageCode Message::getMessageCode()
```

```
{
```

```
    return m_messageCode;
```

```
}
```

```
} /* namespace cea2045 */
```

File: ./cea2045/cea2045/device/message/SetAdvancedLoadUp.cpp

```
#include "SetAdvancedLoadUp.h"
```

```
namespace cea2045 {
```

```
SetAdvancedLoadUp::SetAdvancedLoadUp(unsigned short duration, unsigned short value,  
                                     unsigned char units, unsigned char  
suggestedEfficiency,
```

```
                                     unsigned int eventID, unsigned int startTime,
```

```
                                     unsigned char startRandomization,
```

```
                                     unsigned char endRandomization) :
```

```
    Message(MessageCode::ADVANCED_LOAD_UP_REQUEST)
```

```
{
```

```
    m_msg.header.msgType1 = INTERMEDIATE_MSG_TYP1;
```

```
    m_msg.header.msgType2 = INTERMEDIATE_MSG_TYP2;
```

```
    m_msg.opCode1 = 0x0C; // Advanced Load Up opcode
```

```
    m_msg.opCode2 = 0x00;
```

```
    m_msg.duration = htobe16(duration);
```

```
    m_msg.value = htobe16(value);
```

```
    m_msg.units = units;
```

```
    m_msg.suggestedEfficiency = suggestedEfficiency;
```

```
    m_msg.eventID = htobe32(eventID);
```

```
    m_msg.startTime = htobe32(startTime);
```

```
    m_msg.startRandomization = startRandomization;
```

```
    m_msg.endRandomization = endRandomization;
```

```
    m_msg.setLength();
```



```

        m_msg.setChecksum();
    }

SetAdvancedLoadUp::~SetAdvancedLoadUp()

{

}

int SetAdvancedLoadUp::getNumBytes()

{

    return sizeof(m_msg);

}

unsigned char* SetAdvancedLoadUp::getBuffer()

{

    return (unsigned char *)&m_msg;

}

} /* namespace cea2045 */

```

File: ../cea2045/cea2045/device/message/SetSetpoints.cpp

```

/*
 * SetSetpoint.cpp
 *
 * Created on: Apr 7, 2016
 * Author: dupes
 */

```

```
#include "../message/SetSetpoints.h"
```

```
namespace cea2045
```

```
{
```

```
SetSetpoints::SetSetpoints(unsigned short deviceType, TemperatureUnits units, unsigned  
short setpoint1, unsigned short setpoint2) :
```

```
    Message(MessageCode::SET_SETPOINTS_REQUEST)
```

```
{
```

```
    m_msg.msgType1 = INTERMEDIATE_MSG_TYP1;
```

```
    m_msg.msgType2 = INTERMEDIATE_MSG_TYP2;
```

```
    m_msg.setLength();
```

```
    m_msg.opCode1 = GET_SET;
```

```
    m_msg.opCode2 = TEMP_SETPOINT;
```

```
    m_msg.setDeviceType(deviceType);
```

```
    m_msg.units = (unsigned char)units;
```

```
    m_msg.setSetpoint1(setpoint1);
```

```
    m_msg.setSetpoint2(setpoint2);
```

```
    m_msg.setChecksum();
```

```
}
```

```
SetSetpoints::~~SetSetpoints()
```

```
{
```

```
}
```

```
int SetSetpoints::getNumBytes()
```

```
{
```

```
    return sizeof(m_msg);
```

```
}
```

```
unsigned char* SetSetpoints::getBuffer()
```

```
{
```

```
    return (unsigned char *)&m_msg;
```

```
}
```

```
} /* namespace cea2045 */
```

File: ../cea2045/cea2045/device/message/SetTemperatureOffset.h

```
/*
```

```
 * SetTemperatureOffset.h
```

```
 *
```

```
 * Created on: Apr 7, 2016
```

```
 * Author: dupes
```

```
 */
```

```
#ifndef CEA2045_CEA2045_DEVICE_MESSAGE_SETTEMPERATUREOFFSET_H_
```

```
#define CEA2045_CEA2045_DEVICE_MESSAGE_SETTEMPERATUREOFFSET_H_
```

```
#include "../message/CEA2045MessageSetTemperatureOffsetRequest.h"
```

```
#include "../message/Message.h"
```

```

namespace cea2045

{

class SetTemperatureOffset: public Message
{
private:

    cea2045SetTemperateOffsetRequest m_msg;

public:

    SetTemperatureOffset(unsigned char temperatureOffset, TemperatureUnits units);

    virtual ~SetTemperatureOffset();

    virtual int getNumBytes();

    virtual unsigned char *getBuffer();

};

} /* namespace cea2045 */

#endif /* CEA2045_CEA2045_DEVICE_MESSAGE_SETTEMPERATUREOFFSET_H_ */

```

File: ./cea2045/cea2045/device/message/QueryMessageTypeSupported.h

```

/*

* QueryMessageTypeSupported.h

*

* Created on: Apr 6, 2016

* Author: dupes

```

```
*/

#ifndef CEA2045_CEA2045_DEVICE_MESSAGE_QUERYMESSAGETYPESUPPORTED_H_
#define CEA2045_CEA2045_DEVICE_MESSAGE_QUERYMESSAGETYPESUPPORTED_H_

#include "../message/CEA2045MessageTypeSupported.h"
#include "../message/Message.h"

namespace cea2045

{

class QueryMessageTypeSupported : public Message
{
private:
    cea2045QuerySupport m_msg;

public:
    QueryMessageTypeSupported(MessageCode messageCode, unsigned char messageType1, unsigned
char messageType2);

    virtual ~QueryMessageTypeSupported();

    virtual int getNumBytes();

    virtual unsigned char *getBuffer();
};

} /* namespace cea2045 */
```

```
#endif /* CEA2045_CEA2045_DEVICE_MESSAGE_QUERYMESSAGETYPESUPPORTED_H_ */
```

File: ../cea2045/cea2045/device/message/SetTemperatureOffset.cpp

```
/*
```

```
 * SetTemperatureOffset.cpp
```

```
 *
```

```
 * Created on: Apr 7, 2016
```

```
 * Author: dupes
```

```
 */
```

```
#include "../message/SetTemperatureOffset.h"
```

```
namespace cea2045
```

```
{
```

```
SetTemperatureOffset::SetTemperatureOffset(unsigned char temperatureOffset,
```

```
TemperatureUnits units) :
```

```
Message(MessageCode::SET_TEMPERATURE_OFFSET_REQUEST)
```

```
{
```

```
    m_msg.msgType1 = INTERMEDIATE_MSG_TYP1;
```

```
    m_msg.msgType2 = INTERMEDIATE_MSG_TYP2;
```

```
    m_msg.setLength();
```

```
    m_msg.opCode1 = GET_SET;
```

```
    m_msg.opCode2 = TEMP_OFFSET;
```

```
    m_msg.temperatureOffset = temperatureOffset;
```

```
    m_msg.units = (unsigned char)units;
```

```
m_msg.setChecksum();
```

```
}
```

```
SetTemperatureOffset::~SetTemperatureOffset()
```

```
{
```

```
}
```

```
int SetTemperatureOffset::getNumBytes()
```

```
{
```

```
    return sizeof(m_msg);
```

```
}
```

```
unsigned char* SetTemperatureOffset::getBuffer()
```

```
{
```

```
    return (unsigned char *)&m_msg;
```

```
}
```

```
} /* namespace cea2045 */
```

File: ./cea2045/cea2045/device/message/SetSetpoints.h

```
/*
```

```
 * SetSetpoint.h
```

```
 *
```

```
 * Created on: Apr 7, 2016
```

```

*      Author: dupes

*/

#ifndef CEA2045_CEA2045_DEVICE_MESSAGE_SETSETPOINTS_H_
#define CEA2045_CEA2045_DEVICE_MESSAGE_SETSETPOINTS_H_

#include "../message/CEA2045MessageSetSetpointsRequest.h"
#include "../message/Message.h"

namespace cea2045

{

class SetSetpoints: public Message
{
private:
    cea2045SetSetpointsRequest m_msg;

public:
    SetSetpoints(unsigned short deviceType, TemperatureUnits units, unsigned short
setpoint1, unsigned short setpoint2);
    virtual ~SetSetpoints();

    virtual int getNumBytes();
    virtual unsigned char *getBuffer();
};

} /* namespace cea2045 */

```



```
#endif /* CEA2045_CEA2045_DEVICE_MESSAGE_SETSETPOINTS_H_ */
```

File: ./cea2045/cea2045/processmessage/ProcessMessageSGD.cpp

```
/*
```

```
 * ProcessMessageSGD.cpp
```

```
 *
```

```
 * Created on: Aug 24, 2015
```

```
 * Author: dupes
```

```
 */
```

```
#include "ProcessMessageSGD.h"
```

```
#include "../util/Checksum.h"
```

```
#include "../message/ConvertEnums.h"
```

```
#include "../message/CEA2045MessageIntermediate.h"
```

```
#include "../message/CEA2045MessageBasic.h"
```

```
namespace cea2045 {
```

```
ProcessMessageSGD::ProcessMessageSGD() : m_currentState(0) // Initialize to Idle Normal
```

```
{
```

```
}
```

```
ProcessMessageSGD::~~ProcessMessageSGD()
```

```
{
```

```
}
```

```

void      ProcessMessageSGD::processLinkLayerAckNak(ILinkLayerCommSend      *linkLayer,
cea2045MessageHeader *message)

{

}


void      ProcessMessageSGD::processMessageTypeSupported(ILinkLayerCommSend      *linkLayer,
cea2045MessageHeader *message)

{

}


void ProcessMessageSGD::processBasicMessage(ILinkLayerCommSend *linkLayer, cea2045Basic
*basic)

{
    if (basic->opCode1 == OPER_STATE_REQ) {
        linkLayer->sendLinkLayerAck();

        cea2045Basic response;

        response.msgType1 = BASIC_MSG_TYP1;
        response.msgType2 = BASIC_MSG_TYP2;
        response.opCode1 = OPER_STATE_RESP;
        response.opCode2 = m_currentState;

        linkLayer->sendResponse((unsigned char*)&response, sizeof(response));
    }

    else if (basic->opCode1 == END_SHED) {
        m_currentState = 0;

        linkLayer->sendLinkLayerAck();
    }
}

```

```

    }

}

void ProcessMessageSGD::processIntermediateMessage(ILinkLayerCommSend *linkLayer,
cea2045MessageHeader *message)
{
    cea2045Intermediate *intermediate = (cea2045Intermediate *)message;

    if (intermediate->opCode1 == 0x0C && intermediate->opCode2 == 0x00) {
        linkLayer->sendLinkLayerAck();

        cea2045IntermediateResponse response;

        response.msgType1 = INTERMEDIATE_MSG_TYP1;
        response.msgType2 = INTERMEDIATE_MSG_TYP2;
        response.opCode1 = 0x0C; // Advanced Load Up
        response.opCode2 = 0x80; // Response bit set
        response.responseCode = 0x00; // Success

        linkLayer->sendResponse((unsigned char*)&response, sizeof(response));

        m_currentState = 3; // Running Heightened

        cea2045Basic stateResponse;

        stateResponse.msgType1 = BASIC_MSG_TYP1;
        stateResponse.msgType2 = BASIC_MSG_TYP2;
        stateResponse.opCode1 = OPER_STATE_RESP;
        stateResponse.opCode2 = m_currentState;
    }
}

```

```

        linkLayer->sendResponse((unsigned char*)&stateResponse, sizeof(stateResponse));
    }
}

void          ProcessMessageSGD::processDataLinkMessage(ILinkLayerCommSend      *linkLayer,
cea2045MessageHeader *message)

{

}

void          ProcessMessageSGD::processInvalidMessage(ILinkLayerCommSend      *linkLayer,
cea2045MessageHeader *message)

{

}

} /* namespace cea2045 */

```

File: ./cea2045/cea2045/processmessage/ProcessMessageSGD.h

```

/*
 * ProcessMessageSGD.h
 *
 * Created on: Aug 24, 2015
 * Author: dupes
 */

#ifndef CEA2045_CEA2045_PROCESSMESSAGE_PROCESSMESSAGESGD_H_
#define CEA2045_CEA2045_PROCESSMESSAGE_PROCESSMESSAGESGD_H_

```

```
#include "IProcessMessage.h"
```

```
namespace cea2045 {
```

```
class ProcessMessageSGD : public IProcessMessage
```

```
{
```

```
private:
```

```
    unsigned char m_currentState; // Tracks current operational state
```

```
public:
```

```
    ProcessMessageSGD();
```

```
    virtual ~ProcessMessageSGD();
```

```
        virtual void processLinkLayerAckNak(ILinkLayerCommSend *linkLayer,  
cea2045MessageHeader *message);
```

```
        virtual void processMessageTypeSupported(ILinkLayerCommSend *linkLayer,  
cea2045MessageHeader *message);
```

```
        virtual void processBasicMessage(ILinkLayerCommSend *linkLayer, cea2045Basic  
*basic);
```

```
        virtual void processIntermediateMessage(ILinkLayerCommSend *linkLayer,  
cea2045MessageHeader *message);
```

```
        virtual void processDataLinkMessage(ILinkLayerCommSend *linkLayer,  
cea2045MessageHeader *message);
```

```
        virtual void processInvalidMessage(ILinkLayerCommSend *linkLayer,  
cea2045MessageHeader *message);
```

```
};
```

```
} /* namespace cea2045 */
```

```
#endif /* CEA2045_CEA2045_PROCESSMESSAGE_PROCESSMESSAGESGD_H_ */
```

File: ./cea2045/cea2045/processmessage/IUCM.h

```
/*  
  
 * IUCM.h  
  
 *  
  
 * Created on: Aug 24, 2015  
  
 * Author: dupes  
  
 */  
  
#ifndef CEA2045_CEA2045_PROCESSMESSAGE_IUCM_H_  
#define CEA2045_CEA2045_PROCESSMESSAGE_IUCM_H_  
  
#include "../message/ConvertEnums.h"  
  
#include "../message/CEA2045MessageDeviceInfoResponse.h"  
  
#include "../message/CEA2045MessageCommodityResponse.h"  
  
#include "../message/CEA2045MessageBasic.h"  
  
#include "../message/CEA2045MessageIntermediateResponse.h"  
  
#include "../message/CEA2045MessageGetTemperatureOffsetResponse.h"  
  
#include "../message/CEA2045MessageGetSetpointsResponse.h"  
  
#include "../message/CEA2045MessageGetPresentTemperatureResponse.h"  
  
#include "../message/CEA2045MessageGetUTCTimeResponse.h"  
  
namespace cea2045 {
```

```

class IUCM
{
public:
    IUCM() {};

    virtual ~IUCM() {};

    virtual bool isMessageTypeSupported(MessageTypeCode messageType) = 0;

    virtual MaxPayloadLengthCode getMaxPayload() = 0;

    virtual void processMaxPayloadResponse(MaxPayloadLengthCode maxPayload) = 0;

    virtual void processDeviceInfoResponse(cea2045DeviceInfoResponse *message) = 0;

    virtual void processCommodityResponse(cea2045CommodityResponse *message) = 0;

    virtual void processSetEnergyPriceResponse(cea2045IntermediateResponse *message) = 0;

    virtual void processSetTemperatureOffsetResponse(cea2045IntermediateResponse *message)
= 0;

    virtual void processGetTemperatureOffsetResponse(cea2045GetTemperateOffsetResponse
*message) = 0;

    virtual void processGetSetpointsResponse(cea2045GetSetpointsResponse2 *message) = 0;

    virtual void processGetSetpointsResponse(cea2045GetSetpointsResponse1 *message) = 0;

    virtual void processSetSetpointsResponse(cea2045IntermediateResponse *message) = 0;

    virtual void processStartCyclingResponse(cea2045IntermediateResponse *message) = 0;

    virtual void processTerminateCyclingResponse(cea2045IntermediateResponse *message) = 0;

    virtual void processGetPresentTemperatureResponse(cea2045GetPresentTemperatureResponse
*message) = 0;

    virtual void processGetUTCTimeResponse(cea2045GetUTCTimeResponse *message) = 0;

    virtual void processAckReceived(MessageCode messageCode) = 0;

```

```

virtual void processNakReceived(LinkLayerNakCode nak, MessageCode messageCode) = 0;

virtual void processAppAckReceived(cea2045Basic *message) = 0;
virtual void processAppNakReceived(cea2045Basic *message) = 0;
virtual void processOperationalStateReceived(cea2045Basic *message) = 0;
virtual void processAppCustomerOverride(cea2045Basic *message) = 0;

virtual void processIncompleteMessage(const unsigned char *buffer, unsigned int
numBytes) = 0;

};

} /* namespace cea2045 */

#endif /* CEA2045_CEA2045_PROCESSMESSAGE_IUCM_H_ */

```

File: ../cea2045/cea2045/processmessage/IProcessMessage.h

```

/*
 * ProcessMessage.h
 *
 * Created on: Aug 25, 2015
 * Author: dupes
 */

#ifndef CEA2045_CEA2045_PROCESSMESSAGE_IPROCESSMESSAGE_H_
#define CEA2045_CEA2045_PROCESSMESSAGE_IPROCESSMESSAGE_H_

#include "../linklayer/ILinkLayerCommSend.h"

```



```

#include "../message/CEA2045Message.h"

#include "../message/CEA2045MessageIntermediate.h"

#include "../message/CEA2045MessageIntermediateResponse.h"

#include "../message/CEA2045MessageDataLink.h"

#include "../message/CEA2045MessageBasic.h"

#include "../message/CEA2045MessageSetTemperatureOffsetRequest.h"

#include "../message/CEA2045MessageGetUTCTimeResponse.h"


namespace cea2045 {

class IProcessMessage {

public:

    IProcessMessage() {};

    virtual ~IProcessMessage() {};


    virtual void processLinkLayerAckNak(ILinkLayerCommSend *linkLayer, cea2045MessageHeader
    *message, MessageCode messageCode) = 0;


    virtual void processMessageTypeSupported(ILinkLayerCommSend *linkLayer,
    cea2045MessageHeader *message) = 0;


    virtual void processBasicMessage(ILinkLayerCommSend *linkLayer, cea2045Basic *basic) =
    0;

    virtual void processIntermediateMessage(ILinkLayerCommSend *linkLayer,
    cea2045MessageHeader *message) = 0;

    virtual void processDataLinkMessage(ILinkLayerCommSend *linkLayer, cea2045MessageHeader
    *message) = 0;

```

```
virtual void processInvalidMessage(ILinkLayerCommSend *linkLayer, cea2045MessageHeader
*message) = 0;
```

```
virtual void processIncompleteMessage(ILinkLayerCommSend *linkLayer, const unsigned
char *buffer, unsigned int numBytes) = 0;

};
```

```
} /* namespace cea2045 */
```

```
#endif /* CEA2045_CEA2045_PROCESSMESSAGE_IPROCESSMESSAGE_H_ */
```

File: ./cea2045/cea2045/processmessage/ProcessMessageUCM.cpp

```
/*
 * ProcessMessageUCM.cpp
 *
 * Created on: Aug 24, 2015
 * Author: dupes
 */

#include "ProcessMessageUCM.h"

#include "../util/Checksum.h"

#include <iostream>

#include <iomanip>

namespace cea2045 {

ProcessMessageUCM::ProcessMessageUCM(IUCM *ucm) :
```

```
m_ucm(ucm)
```

```
{  
}
```

```
ProcessMessageUCM::~~ProcessMessageUCM()
```

```
{  
}
```

```
void ProcessMessageUCM::processLinkLayerAckNak(ILinkLayerCommSend *linkLayer,  
cea2045MessageHeader *message, MessageCode messageCode)
```

```
{  
    if (message->msgType1 == LINK_LAYER_ACK_MSG_TYP1)  
    {  
        m_ucm->processAckReceived(messageCode);  
    }  
    else  
    {  
        LinkLayerNakCode nak = ConvertEnums::convertLinkLayerNak(message->msgType2);  
  
        m_ucm->processNakReceived(nak, messageCode);  
    }  
}
```

```
void ProcessMessageUCM::processMessageTypeSupported(ILinkLayerCommSend *linkLayer,
```

```

cea2045MessageHeader *message)

{

    unsigned short messageType = *((unsigned short *)(message));

    MessageTypeCode messageTypeCode = ConvertEnums::convertMessageType(messageType);

    bool messageTypeIsSupported = m_ucm->isMessageTypeSupported(messageTypeCode);

    if (messageTypeIsSupported)

    {

        linkLayer->sendLinkLayerAck();

    }

    else

    {

        linkLayer->sendLinkLayerNak(LinkLayerNakCode::UNSUPPORTED_MESSAGE_TYPE);

    }

}


void ProcessMessageUCM::processBasicMessage(ILinkLayerCommSend *linkLayer, cea2045Basic
*basic)

{

    switch (basic->opCode1)

    {

        case APP_ACK:

            m_ucm->processAppAckReceived(basic);


```

```
linkLayer->sendLinkLayerAck();
```

```
break;
```

```
case APP_NAK:
```

```
m_ucm->processAppNakReceived(basic);
```

```
linkLayer->sendLinkLayerAck();
```

```
break;
```

```
case OPER_STATE_RESP:
```

```
m_ucm->processOperationalStateReceived(basic);
```

```
linkLayer->sendLinkLayerAck();
```

```
break;
```

```
case CUST_OVERRIDE:
```

```
m_ucm->processAppCustomerOverride(basic);
```

```
linkLayer->sendLinkLayerAck();
```

```
cea2045Basic response;
```

```
response.msgType1 = BASIC_MSG_TYP1;
```

```
response.msgType2 = BASIC_MSG_TYP2;
```

```

    response.length = htobe16(2);

    response.opCode1 = APP_ACK;

    response.opCode2 = CUST_OVERRIDE;


    response.checksum = Checksum::calculate((unsigned char *)&response, sizeof(response)
- 2);


    linkLayer->sendResponse((unsigned char*)&response, sizeof(response));


    break;


default:

    linkLayer->sendLinkLayerNak(LinkLayerNakCode::UNSUPPORTED_MESSAGE_TYPE);

    break;

}

}

```

```

void      ProcessMessageUCM::processIntermediateMessage(ILinkLayerCommSend      *linkLayer,
cea2045MessageHeader *message)

{

    if (!m_ucm->isMessageTypeSupported(MessageTypeCode::INTERMEDIATE))

    {

        linkLayer->sendLinkLayerNak(LinkLayerNakCode::REQUEST_NOT_SUPPORTED);

        return;

    }

}

```

```
cea2045Intermediate *intermediate = (cea2045Intermediate *)message;
```

```
unsigned short intermediateType = *((unsigned short *)&(intermediate->opCode1));
```

```
IntermediateTypeCode          intermediateTypeCode          =
```

```
ConvertEnums::convertIntermediateType(intermediateType);
```

```
switch (intermediateTypeCode)
```

```
{
```

```
case IntermediateTypeCode::GET_UTC_TIME_REQUEST:
```

```
{
```

```
if (message->getLength() == 2)
```

```
{
```

```
linkLayer->sendLinkLayerAck();
```

```
cea2045GetUTCTimeResponse response;
```

```
response.utcSeconds = 0;
```

```
response.timezoneOffsetQuarterHours = 0;
```

```
response.dstOffsetQuarterHours = 0;
```

```
m_ucm->processGetUTCTimeResponse(&response);
```

```
response.msgType1 = INTERMEDIATE_MSG_TYP1;
```

```
response.msgType2 = INTERMEDIATE_MSG_TYP2;
```

```
response.setLength();
```

```
response.opCode1 = GET_UTC_TIME;
```

```

response.opCode2 = OP_CODE2_REPLY;

response.responseCode = OP_CODE2_REPLY;


response.setChecksum();


linkLayer->sendResponse((unsigned char*)&response, sizeof(response));
}
else
{
    linkLayer->sendLinkLayerNak(LinkLayerNakCode::REQUEST_NOT_SUPPORTED);
}


break;
}


case IntermediateTypeCode::INFO_RESPONSE:
{
    cea2045DeviceInfoResponse *infoResponse = (cea2045DeviceInfoResponse *)message;

    m_ucm->processDeviceInfoResponse(infoResponse);


    std::cout << "Device Capabilities:" << std::endl;


    uint32_t capabilities = (infoResponse->capability[0] << 24) |

        (infoResponse->capability[1] << 16) |

        (infoResponse->capability[2] << 8) |

```



```

infoResponse->capability[3];

if (capabilities & (1 << 0))

    std::cout << "- Cycling supported" << std::endl;

if (capabilities & (1 << 1))

    std::cout << "- Tier mode supported" << std::endl;

if (capabilities & (1 << 2))

    std::cout << "- Price mode supported" << std::endl;

if (capabilities & (1 << 3))

    std::cout << "- Temperature Offset supported" << std::endl;

if (capabilities & (1 << 4))

    std::cout << "- Continuously variable power" << std::endl;

if (capabilities & (1 << 5))

    std::cout << "- Discretely variable power" << std::endl;

if (capabilities & (1 << 6))

    std::cout << "- Advanced Load Up supported" << std::endl;

if (capabilities & (1 << 7))

    std::cout << "- Price Stream supported" << std::endl;

if (capabilities & (1 << 8))

    std::cout << "- SGD Efficiency Level supported" << std::endl;


std::cout << "Raw capability bytes: "

    << std::hex

    << (int)infoResponse->capability[0] << " "

    << (int)infoResponse->capability[1] << " "

    << (int)infoResponse->capability[2] << " "

    << (int)infoResponse->capability[3]

```

```
<< std::dec << std::endl;
```

```
linkLayer->sendLinkLayerAck();
```

```
break;
```

```
}
```

```
case IntermediateTypeCode::COMMODITY_RESPONSE:
```

```
{
```

```
cea2045CommodityResponse *commodityResponse = (cea2045CommodityResponse *)message;
```

```
m_ucm->processCommodityResponse(commodityResponse);
```

```
linkLayer->sendLinkLayerAck();
```

```
break;
```

```
}
```

```
case IntermediateTypeCode::SET_ENERGY_PRICE_RESPONSE:
```

```
{
```

```
cea2045IntermediateResponse *intermediateResponse = (cea2045IntermediateResponse *)message;
```

```
m_ucm->processSetEnergyPriceResponse(intermediateResponse);
```

```
linkLayer->sendLinkLayerAck();
```

```

        break;
    }

    case IntermediateTypeCode::GET_SET_TEMPERATURE_OFFSET_RESPONSE:
    {
        if (message->getLength() + 6 == sizeof(cea2045IntermediateResponse))
        {
            cea2045IntermediateResponse *intermediateResponse = (cea2045IntermediateResponse
*)message;

            m_ucm->processSetTemperatureOffsetResponse(intermediateResponse);

        }

        else if (message->getLength() + 6 == sizeof(cea2045GetTemperateOffsetResponse))
        {
            cea2045GetTemperateOffsetResponse *getTemperateOffsetResponse =
(cea2045GetTemperateOffsetResponse *)message;

            m_ucm->processGetTemperatureOffsetResponse(getTemperateOffsetResponse);

        }

        else
        {

        }

        linkLayer->sendLinkLayerAck();
    }

```

```

        break;
    }

    case IntermediateTypeCode::GET_SET_SETPOINT_RESPONSE:
    {
        if (message->getLength() + 6 == sizeof(cea2045IntermediateResponse))
        {
            cea2045IntermediateResponse *intermediateResponse = (cea2045IntermediateResponse
*)message;

            m_ucm->processSetSetpointsResponse(intermediateResponse);

            linkLayer->sendLinkLayerAck();
        }
        else if (message->getLength() + 6 == sizeof(cea2045GetSetpointsResponse2))
        {
            cea2045GetSetpointsResponse2 *setpointsResponse = (cea2045GetSetpointsResponse2
*)message;

            m_ucm->processGetSetpointsResponse(setpointsResponse);

            linkLayer->sendLinkLayerAck();
        }
        else if (message->getLength() + 6 == sizeof(cea2045GetSetpointsResponse1))
        {
            cea2045GetSetpointsResponse1 *setpointsResponse = (cea2045GetSetpointsResponse1
*)message;

```

```

m_ucm->processGetSetpointsResponse(setpointsResponse);

linkLayer->sendLinkLayerAck();

}

break;

}

case IntermediateTypeCode::GET_PRESENT_TEMPERATURE_RESPONSE:

{

    cea2045GetPresentTemperatureResponse *getPresentTemperature =
(cea2045GetPresentTemperatureResponse *)message;

    m_ucm->processGetPresentTemperatureResponse(getPresentTemperature);

    linkLayer->sendLinkLayerAck();

    break;

}

case IntermediateTypeCode::START_CYCLING_RESPONSE:

{

    cea2045IntermediateResponse *intermediateResponse = (cea2045IntermediateResponse
*)message;

    m_ucm->processStartCyclingResponse(intermediateResponse);

```

```

linkLayer->sendLinkLayerAck();

break;
}

case IntermediateTypeCode::TERMINATE_CYCLING_RESPONSE:
{
    cea2045IntermediateResponse *intermediateResponse = (cea2045IntermediateResponse
*)message;

    m_ucm->processTerminateCyclingResponse(intermediateResponse);

    linkLayer->sendLinkLayerAck();

    break;
}

default:
    linkLayer->sendLinkLayerNak(LinkLayerNakCode::REQUEST_NOT_SUPPORTED);

    break;
}
}

void ProcessMessageUCM::processDataLinkMessage(ILinkLayerCommSend *linkLayer,
cea2045MessageHeader *message)

```

```

{
    if (!m_ucm->isMessageTypeSupported(MessageTypeCode::DATA LINK))
    {
        linkLayer->sendLinkLayerNak(LinkLayerNakCode::REQUEST_NOT_SUPPORTED);

        return;
    }

    cea2045DataLink *dataLink = (cea2045DataLink *)message;

    DataLinkTypeCode messageType = ConvertEnums::convertDataLinkType(dataLink->opCode1);

    switch (messageType)
    {
        case DataLinkTypeCode::MAX_PAYLOAD_REQUEST:

            linkLayer->sendLinkLayerAck();

            cea2045DataLink response;

            response.msgType1 = DATA LINK_MSG_TYP1;
            response.msgType2 = DATA LINK_MSG_TYP2;
            response.length = htobe16(2);
            response.opCode1 = MAXPAYLOAD_RESP;
            response.opCode2 = (unsigned char)m_ucm->getMaxPayload();

            response.checksum = Checksum::calculate((unsigned char *)&response, sizeof(response) -
2);

```

```

linkLayer->sendResponse((unsigned char*)&response, sizeof(response));

break;

case DataLinkTypeCode::MAX_PAYLOAD_RESPONSE:

    MaxPayloadLengthCode maxPayloadLength;

    maxPayloadLength = ConvertEnums::convertMaxPayloadLength(dataLink->opCode2);

    m_ucm->processMaxPayloadResponse(maxPayloadLength);

    linkLayer->sendLinkLayerAck();

    break;

default:

    linkLayer->sendLinkLayerNak(LinkLayerNakCode::REQUEST_NOT_SUPPORTED);

}

}

void ProcessMessageUCM::processInvalidMessage(ILinkLayerCommSend *linkLayer,
cea2045MessageHeader *message)

{

    linkLayer->sendLinkLayerNak(LinkLayerNakCode::UNSUPPORTED_MESSAGE_TYPE);

}

```



```

void ProcessMessageUCM::processIncompleteMessage(ILinkLayerCommSend *linkLayer, const
unsigned char *buffer, unsigned int numBytes)
{
    m_ucm->processIncompleteMessage(buffer, numBytes);

    linkLayer->sendLinkLayerNak(LinkLayerNakCode::MESSAGE_TIMEOUT);
}

} /* namespace cea2045 */

```

File: ./cea2045/cea2045/processmessage/ProcessMessageUCM.h

```

/*
 * ProcessMessageUCM.h
 *
 * Created on: Aug 24, 2015
 * Author: dupes
 */

#ifndef CEA2045_CEA2045_PROCESSMESSAGE_PROCESSMESSAGEUCM_H_
#define CEA2045_CEA2045_PROCESSMESSAGE_PROCESSMESSAGEUCM_H_

#include "IUCM.h"

#include "../message/ConvertEnums.h"

#include "IProcessMessage.h"

```

```
namespace cea2045 {

class ProcessMessageUCM : public IProcessMessage
{
private:

    IUCM *m_ucm;

public:

    ProcessMessageUCM(IUCM *ucm);

    virtual ~ProcessMessageUCM();

    virtual void processLinkLayerAckNak(ILinkLayerCommSend *linkLayer, cea2045MessageHeader
    *message, MessageCode messageCode);

    virtual void processMessageTypeSupported(ILinkLayerCommSend *linkLayer,
    cea2045MessageHeader *message);

    virtual void processBasicMessage(ILinkLayerCommSend *linkLayer, cea2045Basic *basic);

    virtual void processIntermediateMessage(ILinkLayerCommSend *linkLayer,
    cea2045MessageHeader *message);

    virtual void processDataLinkMessage(ILinkLayerCommSend *linkLayer, cea2045MessageHeader
    *message);

    virtual void processInvalidMessage(ILinkLayerCommSend *linkLayer, cea2045MessageHeader
    *message);
```

```
virtual void processIncompleteMessage(ILinkLayerCommSend *linkLayer, const unsigned
char *buffer, unsigned int numBytes);

};
```

```
} /* namespace cea2045 */
```

```
#endif /* CEA2045_CEA2045_PROCESSMESSAGE_PROCESSMESSAGEUCM_H_ */
```

File: ./cea2045/cea2045/processmessage/ISGD.h

```
/*
 * ISGD.h
 *
 * Created on: Aug 24, 2015
 * Author: dupes
 */
```

```
#ifndef CEA2045_CEA2045_PROCESSMESSAGE_ISGD_H_
```

```
#define CEA2045_CEA2045_PROCESSMESSAGE_ISGD_H_
```

```
namespace cea2045 {
```

```
class ISGD {
```

```
public:
```

```
    ISGD();
```

```
    virtual ~ISGD();
```

```
};
```

```
} /* namespace cea2045 */
```

```
#endif /* CEA2045_CEA2045_PROCESSMESSAGE_ISGD_H_ */
```

File: ./cea2045/cea2045/communicationport/ICommunicationPort.h

```
/*
```

```
 * ICommunicationPort.h
```

```
 *
```

```
 * Created on: Feb 19, 2015
```

```
 * Author: rgause
```

```
*/
```

```
#ifndef ICOMMUNICATIONPORT_H_
```

```
#define ICOMMUNICATIONPORT_H_
```

```
class ICommunicationPort
```

```
{
```

```
public:
```

```
    ICommunicationPort(){}
```

```
    virtual ~ICommunicationPort(){}
```

```
    virtual bool open() = 0;
```

```
    virtual void close() = 0;
```

```
virtual int send(const unsigned char *buffer, int length) = 0;

virtual int recv(unsigned char *buffer, int length, int waitMS) = 0;

};
```

```
#endif /* ICOMMUNICATIONPORT_H_ */
```

File: ./cea2045/cea2045/communicationport/CEA2045SerialPort.h

```
/*
```

```
 * CEA2045SerialPort.h
```

```
 *
```

```
 * Created on: Feb 19, 2015
```

```
 * Author: rgause
```

```
 */
```

```
#ifndef CEA2045SERIALPORT_H_
```

```
#define CEA2045SERIALPORT_H_
```

```
#include <iostream>
```

```
#include <string>
```

```
#include <mutex>
```

```
#include <fcntl.h>
```

```
#include <termios.h>
```

```
#include <unistd.h>
```

```
#include <sys/select.h>
```

```
#include "../communicationport/ICommunicationPort.h"
```

```
namespace cea2045 {

class CEA2045SerialPort : public ICommunicationPort
{
private:

    std::string m_serialPort;

    int m_cea2045_fd;

    struct termios m_tio;

    std::mutex m_mutex;

    bool dataAvailable(int timeoutMS);

public:

    CEA2045SerialPort(std::string serialPort);

    virtual ~CEA2045SerialPort();

    bool open();

    void close();

    int send(const unsigned char *buffer, int length);

    int recv(unsigned char *buffer, int length, int waitMS);

};
```

```
}
```

```
#endif /* CEA2045SERIALPORT_H_ */
```

File: ./cea2045/cea2045/communicationport/CEA2045SerialPort.cpp

```
/*
```

```
 * CEA2045SerialPort.cpp
```

```
 *
```

```
 * Created on: Feb 19, 2015
```

```
 * Author: rgause
```

```
*/
```

```
#include "../communicationport/CEA2045SerialPort.h"
```

```
using namespace std;
```

```
namespace cea2045 {
```

```
CEA2045SerialPort::CEA2045SerialPort(string serialPort)
```

```
{
```

```
    m_serialPort = serialPort;
```

```
    m_cea2045_fd = -1;
```

```
}
```

```
/******
```

```
CEA2045SerialPort::~CEA2045SerialPort()
```

```
{  
}
```

```
/******
```

```
bool CEA2045SerialPort::open()
```

```
{
```

```
    unique_lock<mutex> lock(m_mutex);
```

```
    m_cea2045_fd = ::open(m_serialPort.c_str(), O_RDWR | O_NOCTTY | O_NDELAY); //Open in  
non blocking read/write mode
```

```
    if (m_cea2045_fd == -1)
```

```
        return false;
```

```
    tcgetattr(m_cea2045_fd, &m_tio);
```

```
    m_tio.c_cflag = B19200 | CS8 | CLOCAL | CREAD; //<Set baud rate
```

```
    m_tio.c_iflag = IGNPAR;
```

```
    m_tio.c_oflag = 0;
```

```
    m_tio.c_lflag = 0;
```

```
    tcflush(m_cea2045_fd, TCIFLUSH);
```

```
    tcsetattr(m_cea2045_fd, TCSANOW, &m_tio);
```

```
    return true;
```



```
}
```

```
/*****
```

```
void CEA2045SerialPort::close()
```

```
{
```

```
    unique_lock<mutex> lock(m_mutex);
```

```
    ::close(m_cea2045_fd);
```

```
}
```

```
/*****
```

```
int CEA2045SerialPort::send(const unsigned char *buffer, int length)
```

```
{
```

```
    return write(m_cea2045_fd, buffer, length);
```

```
}
```

```
/*****
```

```
bool CEA2045SerialPort::dataAvailable(int timeoutMS)
```

```
{
```

```
    fd_set rfds;
```

```
    struct timeval tv;
```

```
    FD_ZERO(&rfds);
```

```

FD_SET(m_cea2045_fd, &rfds);

tv.tv_sec = 0;

tv.tv_usec = 1000 * timeoutMS;

int retval = select(m_cea2045_fd + 1, &rfd, NULL, NULL, &tv);

if (retval == -1)

    return false;

if (FD_ISSET(m_cea2045_fd, &rfd))

    return true;

return false;
}

/*****/

int CEA2045SerialPort::recv(unsigned char *buffer, int length, int waitMS)
{
    if (dataAvailable(waitMS))
    {
        unique_lock<mutex> lock(m_mutex);

        return read(m_cea2045_fd, buffer, length);
    }
}

```

```
    return 0;

}

} // namespace cea2045
```

File: ../cea2045/cea2045/message/CEA2045MessageGetUTCTimeResponse.h

```
/*
 * CEA2045MessageGetUTCTimeResponse.h
 *
 * Created on: Dec 13, 2016
 * Author: dupes
 */

#ifndef CEA2045_CEA2045_MESSAGE_CEA2045MESSAGEGETUTCTIMERESPONSE_H_
#define CEA2045_CEA2045_MESSAGE_CEA2045MESSAGEGETUTCTIMERESPONSE_H_

#include "CEA2045MessageMacros.h"

namespace cea2045 {

struct cea2045GetUTCTimeResponse
{
    unsigned char msgType1;

    unsigned char msgType2;

    unsigned short length;
}
```

```
unsigned char opCode1;

unsigned char opCode2;

unsigned char responseCode;

unsigned int utcSeconds;

char timezoneOffsetQuarterHours;

char dstOffsetQuarterHours;


unsigned short checksum;


MACRO_LENGTH

MACRO_CHECKSUM

} __attribute__((packed));

}

#endif /* CEA2045_CEA2045_MESSAGE_CEA2045MESSAGEGETUTCTIMERESPONSE_H_ */
```

File: ./cea2045/cea2045/message/CEA2045MessageTerminateCyclingRequest.h

```
/*

 * CEA2045MessageTerminateCycling.h

 *

 * Created on: Apr 8, 2016

 * Author: dupes

 */


#ifndef CEA2045_CEA2045_MESSAGE_CEA2045MESSAGETERMINATECYCLINGREQUEST_H_

#define CEA2045_CEA2045_MESSAGE_CEA2045MESSAGETERMINATECYCLINGREQUEST_H_
```

```
#include "CEA2045MessageMacros.h"

namespace cea2045

{

struct cea2045TerminateCyclingRequest

{

    unsigned char msgType1;

    unsigned char msgType2;

    unsigned short length;

    unsigned char opCode1;

    unsigned char opCode2;

    unsigned int eventID;

    unsigned char endRandomizationInMinutes;

    unsigned short checksum;

    void setEventID(unsigned int value)

    {

        eventID = htobe32(value);

    }

    MACRO_LENGTH

    MACRO_CHECKSUM

} __attribute__((packed));

} /* namespace cea2045 */
```

```
#endif /* CEA2045_CEA2045_MESSAGE_CEA2045MESSAGETERMINATECYCLINGREQUEST_H_ */
```

File: ./cea2045/cea2045/message/CEA2045MessageMacros.h

```
/*
```

```
 * CEA2045MessageMacros.h
```

```
 *
```

```
 * Created on: Apr 8, 2016
```

```
 * Author: dupes
```

```
*/
```

```
#ifndef CEA2045_CEA2045_MESSAGE_CEA2045MESSAGEMACROS_H_
```

```
#define CEA2045_CEA2045_MESSAGE_CEA2045MESSAGEMACROS_H_
```

```
#include <endian.h>
```

```
#include "../util/Checksum.h"
```

```
namespace cea2045
```

```
{
```

```
#define MACRO_LENGTH \
```

```
void setLength() \
```

```
{ \
```

```
    length = htobe16(sizeof(*this) - 6); \
```

```
} \
```

```
unsigned short getLength() \
```

```
{ \
```

```
    return be16toh(length); \
```

```
} \
```

```
#define MACRO_CHECKSUM \
```

```
void setChecksum() \
```

```
{ \
```

```
checksum = Checksum::calculate((unsigned char *)this, sizeof(*this) - 2); \
```

```
} \
```

```
} /* namespace cea2045 */
```

```
#endif /* CEA2045_CEA2045_MESSAGE_CEA2045MESSAGEMACROS_H_ */
```

File: ./cea2045/cea2045/message/ConvertEnums.h

```
/*
```

```
* ConvertEnums.h
```

```
*
```

```
* Created on: Aug 24, 2015
```

```
* Author: dupes
```

```
*/
```

```
#ifndef CEA2045_CEA2045_PROCESSMESSAGE_CONVERTENUMS_H_
```

```
#define CEA2045_CEA2045_PROCESSMESSAGE_CONVERTENUMS_H_
```

```
namespace cea2045 {
```

```
#define BASIC_MESSAGE 0x0801
```

```
#define INTERMEDIATE_MESSAGE 0x0802
```

```
#define DATA_LINK_MESSAGE 0x0803

#define SHED 0x01

#define END_SHED 0x02

#define APP_ACK 0x03

#define APP_NAK 0x04

#define POWER_LEVEL 0x06

#define PRESENT_RELATIVE_PRICE 0x07

#define NEXT_RELATIVE_PRICE 0x08

#define TIME_REMAINING_PRICE 0x09

#define CPP 0x0a

#define GRID_EMERGENCY 0x0b

#define GRID_GUIDANCE 0x0c

#define COMM_STATUS 0x0e

#define CUST_OVERRIDE 0x11

#define OPER_STATE_REQ 0x12

#define OPER_STATE_RESP 0x13

#define SLEEP_STATE 0x14

#define WAKE_STATE 0x15

#define SIMPLE_TIME_SYNC 0x16

#define LOADUP 0x17

#define INFO_REQ 0x01

#define SET.UTC_TIME 0x02

#define GET.UTC_TIME 0x02

#define MSG_TYPE_SUP 0xFE

#define ECHONET_LT 0x08

#define AUTO_CYCLE 0x04
```



```
#define AC_START 0x00

#define AC_STOP 0x01

#define GET_SET 0x03

#define ENERGY_PRICE 0x00

#define TIER 0x01

#define TEMP_OFFSET 0x02

#define TEMP_SETPPOINT 0x03

#define PRESENT_TEMPERATURE 0x04


#define COMMODITY_READ 0x06

#define COMMODITY_CODE_ELECTRICITY_CONSUMED 0x00

#define COMMODITY_CODE_ELECTRICITY_PRODUCED 0x01

#define COMMODITY_CODE_NATURAL_GAS_ENGLISH 0x02

#define COMMODITY_CODE_WATER_ENGLISH 0x03

#define COMMODITY_CODE_NATURAL_GAS_METRIC 0x04

#define COMMODITY_CODE_WATER_METRIC 0x05

#define COMMODITY_CODE_TOTAL_ENERGY_STORAGE 0x06

#define COMMODITY_CODE_PRESENT_ENERGY_STORAGE 0x07


#define MAXPAYLOAD_REQ_OP_CODE1 0x18

#define CLEAR_OP_CODE2 0x00

#define MAXPAYLOAD_RESP 0x19


#define DATALINK_MSG_TYP1 0x08

#define DATALINK_MSG_TYP2 0x03


#define INTERMEDIATE_MSG_TYP1 0x08
```

```
#define INTERMEDIATE_MSG_TYP2 0x02

#define BASIC_MSG_TYP1 0x08

#define BASIC_MSG_TYP2 0x01

#define LINK_LAYER_ACK_MSG_TYP1 0x06

#define LINK_LAYER_ACK_MSG_TYP2 0x00

#define LINK_LAYER_NAK_MSG_TYP1 0x15

#define DEVICE_INFO_RESPONSE_OP_CODE1 0x01

#define DEVICE_INFO_RESPONSE_OP_CODE2 0x81

#define OP_CODE2_REPLY_RESPONSE 0x81

#define OP_CODE2_REPLY 0x80

#define ADVANCED_LOAD_UP 0x0C

enum class MaxPayloadLengthCode
{
    LENGTH2,
    LENGTH4,
    LENGTH8,
    LENGTH16,
    LENGTH32,
    LENGTH64,
    LENGTH128,
```

LENGTH256,

LENGTH512,

LENGTH1024,

LENGTH1280,

LENGTH1500,

LENGTH2048,

LENGTH4096,

INVALID

};

enum class MessageTypeCode

{

NONE,

BASIC,

INTERMEDIATE,

DATALINK

};

enum class MessageCode

{

NONE,

MAX_PAYLOAD_REQUEST,

MAX_PAYLOAD_RESPONSE,

SUPPORT_DATALINK_MESSAGES,

SUPPORT_INTERMEDIATE_MESSAGES,

```
BASIC_CRITICAL_PEAK_EVENT_REQUEST,  
  
BASIC_END_SHED_REQUEST,  
  
BASIC_SHED_REQUEST,  
  
BASIC_GRID_EMERGENCY_REQUEST,  
  
BASIC_LOAD_UP_REQUEST,  
  
BASIC_OUTSIDE_COMM_CONNECTION_STATUS_MESSAGE,  
  
BASIC_PRESENT_RELATIVE_PRICE_REQUEST,  
  
BASIC_NEXT_RELATIVE_PRICE_REQUEST,  
  
BASIC_QUERY_OPERATIONAL_STATE_REQUEST,  
  
BASIC_POWER_LEVEL,  
  
  
  
DEVICE_INFORMATION_REQUEST,  
  
GET_COMMODITY_REQUEST,  
  
GET_TEMPERATURE_OFFSET,  
  
GET_SETPOINTS_REQUEST,  
  
GET_PRESENT_TEMPERATURE_REQUEST,  
  
SET_TEMPERATURE_OFFSET_REQUEST,  
  
SET_SETPOINTS_REQUEST,  
  
SET_ENERGY_PRICE_REQUEST,  
  
ADVANCED_LOAD_UP_REQUEST,  
  
START_CYCLING_REQUEST,  
  
TERMINATE_CYCLING_REQUEST,  
  
  
  
CUSTOMER_OVERRIDE_RESPONSE  
  
};
```

```
enum class LinkLayerNakCode
```

```
{  
  
    NO_REASON,  
  
    INVALID_BYTE,  
  
    INVALID_LENGTH,  
  
    CHECKSUM_ERROR,  
  
    RESERVED,  
  
    MESSAGE_TIMEOUT,  
  
    UNSUPPORTED_MESSAGE_TYPE,  
  
    REQUEST_NOT_SUPPORTED,  
  
    NONE  
  
};
```

```
enum class DataLinkTypeCode
```

```
{  
  
    INVALID,  
  
    MAX_PAYLOAD_REQUEST = 0x18,  
  
    MAX_PAYLOAD_RESPONSE = 0x19,  
  
  
  
    SEND_NEXT_COMMAND_TO_SLOT = 0x1e  
  
};
```

```
enum class IntermediateTypeCode
```

```
{  
  
    INVALID,  
  
    INFO_REQUEST = 0x0101,  
  
    INFO_RESPONSE = 0x0181,  
  
    GET_UTC_TIME_REQUEST = 0x0200,
```

```
COMMODITY_REQUEST = 0x0600,  
  
COMMODITY_RESPONSE = 0x0680,  
  
SET_ENERGY_PRICE_RESPONSE = 0x0380,  
  
GET_SET_TEMPERATURE_OFFSET_REQUEST = 0x0302,  
  
GET_SET_TEMPERATURE_OFFSET_RESPONSE = 0x0382,  
  
GET_SET_SETPPOINT_RESPONSE = 0x0383,  
  
GET_PRESENT_TEMPERATURE_RESPONSE = 0x0384,  
  
START_CYCLING_RESPONSE = 0x0480,  
  
TERMINATE_CYCLING_RESPONSE = 0x0481,  
  
ADVANCED_LOAD_UP_REQUEST = 0x0C00,  
  
    ADVANCED_LOAD_UP_RESPONSE = 0x0C80  
};
```

```
enum class OutsideCommuncatonStatusCode  
  
{  
  
    No,  
  
    Found,  
  
    Poor  
  
};
```

```
enum class TemperatureUnits  
  
{  
  
    F,  
  
    C  
  
};
```

```
enum class DeviceTypes
```

```
{  
  
UNSPECIFIED_TYPE = 0x0000,  
  
WATER_HEATER_GAS = 0x0001,  
  
WATER_HEATER_ELECTRIC = 0x0002,  
  
WATER_HEATER_HEAT_PUMP = 0x0003,  
  
CENTRAL_AC_HEAT_PUMP = 0x0004,  
  
CENTRAL_AC_FOSSIL_FUEL_HEAT = 0x0005,  
  
CENTRAL_AC_RESISTANCE_HEAT = 0x0006,  
  
CENTRAL_AC_ONLY = 0x0007,  
  
EVAPORATIVE_COOLER = 0x0008,  
  
BASEBOARD_ELECTRIC_HEAT = 0x0009,  
  
WINDOW_AC = 0x000A,  
  
PORTABLE_ELECTRIC_HEATER = 0x000B,  
  
CLOTHES_WASHER = 0x000C,  
  
CLOTHES_DRYER_GAS = 0x000D,  
  
CLOTHES_DRYER_ELECTRIC = 0x000E,  
  
REFRIGERATOR_FREEZER = 0x000F,  
  
FREEZER = 0x0010,  
  
DISHWASHER = 0x0011,  
  
MICROWAVE_OVEN = 0x0012,  
  
OVEN_ELECTRIC = 0x0013,  
  
OVEN_GAS = 0x0014,  
  
COOK_TOP_ELECTRIC = 0x0015,  
  
COOK_TOP_GAS = 0x0016,  
  
STOVE_ELECTRIC = 0x0017,  
  
STOVE_GAS = 0x0018,  
  
DEHUMIDIFIER = 0x0019,
```

FAN = 0x0020,
POOL_PUMP_SINGLE_SPEED = 0x0030,
POOL_PUMP_VARIABLE_SPEED = 0x0031,
ELECTRIC_HOT_TUB = 0x0032,
IRRIGATION_PUMP = 0x0040,
ELECTRIC_VEHICLE = 0x1000,
HYBRID_VEHICLE = 0x1001,
ELECTRIC_VEHICLE_SUPPLY_EQUIPMENT_GENERAL = 0x1100,
ELECTRIC_VEHICLE_SUPPLY_EQUIPMENT_LEVEL_1 = 0x1101,
ELECTRIC_VEHICLE_SUPPLY_EQUIPMENT_LEVEL_2 = 0x1102,
ELECTRIC_VEHICLE_SUPPLY_EQUIPMENT_LEVEL_3 = 0x1103,
IN_PREMISES_DISPLAY = 0x2000,
ENERGY_MANAGER = 0x5000,
GATEWAY_DEVICE = 0x6000,
WIRELESS = 0x4000,
PLC = 0x4001,
WIRED = 0x4002,
IEEE_802_15_4 = 0x4003,
IEEE_802_11 = 0x4004,
IEEE_802_16 = 0x4005,
VHF_UHF_PAGER = 0x4006,
FM = 0x4007,
WIRED_ETHERNET = 0x4008,
COAXIAL_NETWORKING = 0x4009,
TELEPHONE_LINE = 0x400A,
IEEE_1901 = 0x400B,
IEEE_1901_2 = 0x400C,


```
ITUT_G_HN = 0x400D,  
  
ITUT_G_HNEM = 0x400E,  
  
CELLULAR = 0x400F,  
  
UTILITY_AMI_WIRELESS = 0x4010,  
  
UTILITY_AMI_PLC = 0x4011  
  
};
```

```
enum class CurrencyCode  
  
{  
  
    US_DOLLAR = 840,  
  
    EURO = 978,  
  
    MEXICAN_PESO = 484,  
  
    CANADIAN_DOLLAR = 124  
  
};
```

```
class ConvertEnums  
  
{  
  
  
  
  
  
  
  
private:  
  
    ConvertEnums();  
  
  
  
  
  
  
  
public:  
  
    virtual ~ConvertEnums();  
  
  
  
  
  
  
  
    static MessageTypeCode convertMessageType(unsigned short messageType);  
  
  
  
  
  
  
  
    static LinkLayerNakCode convertLinkLayerNak(unsigned char nak);
```

```

static DataLinkTypeCode convertDataLinkType(unsigned char dataLinkType);

static MaxPayloadLengthCode convertMaxPayloadLength(unsigned char maxPayloadLength);

static IntermediateTypeCode convertIntermediateType(unsigned short intermediateType);
};

} /* namespace cea2045 */

#endif /* CEA2045_CEA2045_PROCESSMESSAGE_CONVERTENUMS_H_ */

```

File: ../cea2045/cea2045/message/CEA2045Message.h

```

/*
 * CEA2045Message.h
 *
 * Created on: Feb 18, 2015
 *
 * Author: dupes
 */

#ifndef CEA2045MESSAGE_H_
#define CEA2045MESSAGE_H_

#include <endian.h>

#include <iostream>

#include <string.h>

#include "../message/ConvertEnums.h"

```

```
namespace cea2045 {

struct cea2045Link

{
    unsigned char msgType1;
    unsigned char msgType2;
} __attribute__((packed));

struct cea2045MessageHeader

{
    unsigned char msgType1;
    unsigned char msgType2;
    unsigned short length;

    unsigned short getLength()
    {
        return be16toh(length);
    }

    bool isLinkLayerAckNak()
    {
        if (msgType1 == LINK_LAYER_ACK_MSG_TYP1 ||
            msgType1 == LINK_LAYER_NAK_MSG_TYP1)
            return true;

        return false;
    }
}
```

```
}
```

```
bool isLinkLayerNak()
```

```
{
```

```
    if (msgType1 == LINK_LAYER_NAK_MSG_TYP1)
```

```
        return true;
```

```
    return false;
```

```
}
```

```
bool isBasicMessage()
```

```
{
```

```
    if (msgType1 == BASIC_MSG_TYP1 && msgType2 == BASIC_MSG_TYP2)
```

```
        return true;
```

```
    return false;
```

```
}
```

```
bool isDataLinkMessage()
```

```
{
```

```
    if (msgType1 == DATALINK_MSG_TYP1 && msgType2 == DATALINK_MSG_TYP2)
```

```
        return true;
```

```
    return false;
```

```
}
```

```

bool isIntermediateMessage()
{
    if (msgType1 == INTERMEDIATE_MSG_TYP1 && msgType2 == INTERMEDIATE_MSG_TYP2)
        return true;

    return false;
}

} __attribute__((packed));

} // namespace cea2045

#endif /* CEA2045MESSAGE_H_ */

```

File: ./cea2045/cea2045/message/CEA2045MessageDeviceInfoResponse.h

```

/*
 * CEA2045DeviceInfoResponse.h
 *
 * Created on: Mar 9, 2015
 * Author: rgause
 */

#ifndef CEA2045MESSAGEDEVICEINFORESPONSE_H_
#define CEA2045MESSAGEDEVICEINFORESPONSE_H_

#include "CEA2045MessageMacros.h"

```

```
namespace cea2045 {

struct cea2045DeviceInfoResponse

{
    unsigned char msgType1;

    unsigned char msgType2;

    unsigned short length;

    unsigned char opCode1;

    unsigned char opCode2;

    unsigned char responseCode;

    unsigned char version[2];

    unsigned short vendorId;

    unsigned short deviceType;

    unsigned char deviceRevision[2];

    unsigned char capability[4];

    unsigned char reserved1;

    unsigned char modelNumber[16];

    unsigned char serialNumber[16];

    unsigned char firmwareYear20xx;

    unsigned char firmwareMonth;

    unsigned char firmwareDay;

    unsigned char firmwareMajor;

    unsigned char firmwareMinor;

    unsigned short checksum;

    unsigned short getVendorID()

    {
```

```

    return be16toh(vendorId);
}

unsigned short getDeviceType()
{
    return be16toh(deviceType);
}

MACRO_LENGTH

MACRO_CHECKSUM

} __attribute__((packed));

} // cea2045

#endif /* CEA2045MESSAGEDEVICEINFORESPONSE_H_ */

```

File: ./cea2045/cea2045/message/CEA2045MessageStartCyclingRequest.h

```

/*
 * CEA2045MessageStartCyclingRequest.h
 *
 * Created on: Apr 7, 2016
 * Author: dupes
 */

#ifndef CEA2045_CEA2045_MESSAGE_CEA2045MESSAGESTARTCYCLINGREQUEST_H_
#define CEA2045_CEA2045_MESSAGE_CEA2045MESSAGESTARTCYCLINGREQUEST_H_

```

```
#include "CEA2045MessageMacros.h"
```

```
namespace cea2045
```

```
{
```

```
struct cea2045StartCyclingRequest
```

```
{
```

```
    unsigned char msgType1;
```

```
    unsigned char msgType2;
```

```
    unsigned short length;
```

```
    unsigned char opCode1;
```

```
    unsigned char opCode2;
```

```
    unsigned int eventID;
```

```
    unsigned int startTimeUTCSince2000;
```

```
    unsigned short durationInMinutes;
```

```
    unsigned char dutyCycle;
```

```
    unsigned char startRandomizationInMinutes;
```

```
    unsigned char endRandomizationInMinutes;
```

```
    unsigned char criticality;
```

```
    unsigned short checksum;
```

```
void setEventID(unsigned int value)
```

```
{
```

```
    eventID = htobe32(value);
```

```
}
```

```
void setStartTimeUTCSince2000(unsigned int value)
```



```

{

    startTimeUTCSince2000 = htobe32(value);

}


void setDurationInMinutes(unsigned short value)

{

    durationInMinutes = htobe16(value);

}


MACRO_LENGTH

MACRO_CHECKSUM

} __attribute__((packed));


} /* namespace cea2045 */


#endif /* CEA2045_CEA2045_MESSAGE_CEA2045MESSAGESTARTCYCLINGREQUEST_H_ */


File: ./cea2045/cea2045/message/CEA2045MessageGetPresentTemperatureResponse.h

/*

* CEA2045MessageGetPresentTemperatureResponse.h

*

* Created on: Apr 25, 2016

* Author: dupes

*/


#ifndef CEA2045_CEA2045_MESSAGE_CEA2045MESSAGEGETPRESENTTEMPERATURERESPONSE_H_

#define CEA2045_CEA2045_MESSAGE_CEA2045MESSAGEGETPRESENTTEMPERATURERESPONSE_H_


```

```
#include "CEA2045MessageMacros.h"
```

```
namespace cea2045 {
```

```
struct cea2045GetPresentTemperatureResponse
```

```
{
```

```
    unsigned char msgType1;
```

```
    unsigned char msgType2;
```

```
    unsigned short length;
```

```
    unsigned char opCode1;
```

```
    unsigned char opCode2;
```

```
    unsigned char responseCode;
```

```
    unsigned short deviceType;
```

```
    unsigned char units;
```

```
    unsigned short presentTemperature;
```

```
    unsigned short checksum;
```

```
void setDeviceType(unsigned short value)
```

```
{
```

```
    deviceType = htobe16(value);
```

```
}
```

```
void setPresentTemperature(unsigned short value)
```

```
{
```

```
    presentTemperature = htobe16(value);
```

```
}
```

```
unsigned short getDeviceType()
```

```
{
```

```
    return be16toh(deviceType);
```

```
}
```

```
unsigned short getPresentTemperature()
```

```
{
```

```
    return be16toh(presentTemperature);
```

```
}
```

```
MACRO_LENGTH
```

```
MACRO_CHECKSUM
```

```
} __attribute__((packed));
```

```
}
```

```
#endif /* CEA2045_CEA2045_MESSAGE_CEA2045MESSAGEGETPRESENTTEMPERATURERESPONSE_H_ */
```

File: ./cea2045/cea2045/message/CEA2045MessageCommodityResponse.h

```
/*
```

```
 * CEA2045CommodityResponse.h
```

```
 *
```

```
 * Created on: Mar 10, 2015
```

```
 * Author: rgause
```

```
*/
```

```
#ifndef CEA2045MESSAGECOMMODITYRESPONSE_H_

#define CEA2045MESSAGECOMMODITYRESPONSE_H_


#include <endian.h>

#include <stdint.h>

#include "CEA2045MessageMacros.h"


namespace cea2045 {

#define CEA2045_MAX_COMMODITY_RESPONSE_LENGTH 101


struct cea2045CommodityData

{

    unsigned char commodityCode;

    unsigned char instantaneousRate[6];

    unsigned char cumulativeAmount[6];


    int64_t getInstantaneousRate()

    {

        return ((int64_t)instantaneousRate[0] << 40) +

            ((int64_t)instantaneousRate[1] << 32) +

            ((int64_t)instantaneousRate[2] << 24) +

            ((int64_t)instantaneousRate[3] << 16) +

            ((int64_t)instantaneousRate[4] << 8) +

            ((int64_t)instantaneousRate[5]);

    }

}
```

```
int64_t getCumulativeAmount()
{
    return ((int64_t)cumulativeAmount[0] << 40) +
        ((int64_t)cumulativeAmount[1] << 32) +
        ((int64_t)cumulativeAmount[2] << 24) +
        ((int64_t)cumulativeAmount[3] << 16) +
        ((int64_t)cumulativeAmount[4] << 8) +
        ((int64_t)cumulativeAmount[5]);
}
};
```

```
struct cea2045CommodityResponse
{
    unsigned char msgType1;
    unsigned char msgType2;
    unsigned short length;
    unsigned char opCode1;
    unsigned char opCode2;
    unsigned char responseCode;

    cea2045CommodityData commodityData[7];

    unsigned short checksum;

    MACRO_LENGTH

    MACRO_CHECKSUM
```

```

unsigned short getCommodityDataCount()

{

    return (getLength() / sizeof(cea2045CommodityData));

}


cea2045CommodityData *getCommodityData(int index)

{

    int count = getCommodityDataCount();

    if (index > count - 1)

        return nullptr;

    unsigned char *buffer = (unsigned char *)&msgType1;

    return (cea2045CommodityData *)((buffer + 7) + index * sizeof(cea2045CommodityData));

}

}__attribute__((packed));

} // namespace cea2045


#endif /* CEA2045MESSAGECOMMODITYRESPONSE_H_ */

```

File: ./cea2045/cea2045/message/CEA2045MessageDataLink.h

```

/*

* CEA2045MessageDataLink.h

*

```

```
*   Created on: Feb 18, 2015

*       Author: rgause

*/


#ifndef CEA2045MESSAGEDATALINK_H_

#define CEA2045MESSAGEDATALINK_H_


#include "CEA2045MessageMacros.h"


namespace cea2045 {


struct cea2045DataLink

{

    unsigned char msgType1;

    unsigned char msgType2;

    unsigned short length;

    unsigned char opCode1;

    unsigned char opCode2;

    unsigned short checksum;


    MACRO_LENGTH

    MACRO_CHECKSUM

} __attribute__((packed));


} // namespace cea2045


#endif /* CEA2045MESSAGEDATALINK_H_ */
```

File: ./cea2045/cea2045/message/CEA2045MessageIntermediateResponse.h

```
/*
 * CEA2045MessageIntermediateResponse.h
 *
 * Created on: Apr 7, 2016
 * Author: dupes
 */

#ifndef CEA2045_CEA2045_MESSAGE_CEA2045MESSAGEINTERMEDIATERESPONSE_H_
#define CEA2045_CEA2045_MESSAGE_CEA2045MESSAGEINTERMEDIATERESPONSE_H_

#include "CEA2045MessageMacros.h"

namespace cea2045
{

struct cea2045IntermediateResponse
{
    unsigned char msgType1;
    unsigned char msgType2;
    unsigned short length;
    unsigned char opCode1;
    unsigned char opCode2;
    unsigned char responseCode;
    unsigned short checksum;

    MACRO_LENGTH
```


MACRO_CHECKSUM

} __attribute__((packed));

} /* namespace cea2045 */

#endif /* CEA2045_CEA2045_MESSAGE_CEA2045MESSAGEINTERMEDIATERESPONSE_H_ */

File: ./cea2045/cea2045/message/CEA2045MessageIntermediate.h

/*

* CEA2045MessageIntermediate.h

*

* Created on: Mar 5, 2015

* Author: rgause

*/

#ifndef CEA2045MESSAGEINTERMEDIATE_H_

#define CEA2045MESSAGEINTERMEDIATE_H_

#include "CEA2045MessageMacros.h"

namespace cea2045 {

struct cea2045Intermediate

{

unsigned char msgType1;

unsigned char msgType2;

unsigned short length;

```
unsigned char opCode1;

unsigned char opCode2;

unsigned short checksum;


MACRO_LENGTH

MACRO_CHECKSUM

} __attribute__((packed));


}


#endif /* CEA2045MESSAGEINTERMEDIATE_H_ */
```

File: ./cea2045/cea2045/message/CEA2045MessageSetEnergyPriceRequest.h

```
/*

 * CEA2045MessageSetEnergyPrice.h

 *

 * Created on: Apr 7, 2016

 * Author: dupes

 */


#ifndef CEA2045_CEA2045_MESSAGE_CEA2045MESSAGESETENERGYPRICEREQUEST_H_
#define CEA2045_CEA2045_MESSAGE_CEA2045MESSAGESETENERGYPRICEREQUEST_H_


#include "CEA2045MessageMacros.h"


namespace cea2045 {
```

```
struct cea2045SetEnergyPriceRequest
{
    unsigned char msgType1;
    unsigned char msgType2;
    unsigned short length;
    unsigned char opCode1;
    unsigned char opCode2;
    unsigned short currentPrice;
    unsigned short currencyCode;
    unsigned char digitsAfterDecimalPoint;
    unsigned int expirationDatetimeUTC;
    unsigned int nextPrice;
    unsigned short checksum;

    void setCurrentPrice(unsigned short value)
    {
        currentPrice = htobe16(value);
    }

    void setCurrencyCode(unsigned short value)
    {
        currencyCode = htobe16(value);
    }

    void setExpirationDatetimeUTC(unsigned int value)
    {
        expirationDatetimeUTC = htobe32(value);
    }
}
```

```

}

void setNextPrice(unsigned int value)

{
    nextPrice = htobe32(value);
}

MACRO_LENGTH

MACRO_CHECKSUM

} __attribute__((packed));

} /* namespace cea2045 */

#endif /* CEA2045_CEA2045_MESSAGE_CEA2045MESSAGESETENERGYPRICEREQUEST_H_ */

```

File: ./cea2045/cea2045/message/CEA2045MessageGetTemperatureOffsetResponse.h

```

/*
 * CEA2045MessageGetTemperatureOffsetResponse.h
 *
 * Created on: Apr 11, 2016
 *
 * Author: dupes
 */

#ifndef CEA2045_CEA2045_MESSAGE_CEA2045MESSAGEGETTEMPERATUREOFFSETRESPONSE_H_
#define CEA2045_CEA2045_MESSAGE_CEA2045MESSAGEGETTEMPERATUREOFFSETRESPONSE_H_

#include "CEA2045MessageMacros.h"

```

```

namespace cea2045 {

    struct cea2045GetTemperateOffsetResponse
    {
        unsigned char msgType1;
        unsigned char msgType2;
        unsigned short length;
        unsigned char opCode1;
        unsigned char opCode2;
        unsigned char responseCode;
        unsigned char currentOffset;
        unsigned char units;
        unsigned short checksum;

        MACRO_LENGTH

        MACRO_CHECKSUM
    } __attribute__((packed));

}

#endif /* CEA2045_CEA2045_MESSAGE_CEA2045MESSAGEGETTEMPERATUREOFFSETRESPONSE_H_ */

```

File: ./cea2045/cea2045/message/CEA2045MessageTypeSupported.h

```

/*
 * CEA2045MessageTypeSupported.h
 *

```

* Created on: Mar 12, 2015

* Author: nebland

*/

#ifndef CEA2045_CEA2045_MESSAGE_CEA2045MESSAGETYPESUPPORTED_H_

#define CEA2045_CEA2045_MESSAGE_CEA2045MESSAGETYPESUPPORTED_H_

#include "CEA2045MessageMacros.h"

namespace cea2045 {

struct cea2045QuerySupport

{

unsigned char msgType1;

unsigned char msgType2;

unsigned short length;

unsigned short checksum;

MACRO_LENGTH

MACRO_CHECKSUM

} __attribute__((packed));

}

#endif /* CEA2045_CEA2045_MESSAGE_CEA2045MESSAGETYPESUPPORTED_H_ */

File: ./cea2045/cea2045/message/CEA2045MessageSetSetpointsRequest.h

```
/*
 * CEA2045MessageSetSetpoints.h
 *
 * Created on: Apr 7, 2016
 * Author: dupes
 */

#ifndef CEA2045_CEA2045_MESSAGE_CEA2045MESSAGESETSETPOINTSREQUEST_H_
#define CEA2045_CEA2045_MESSAGE_CEA2045MESSAGESETSETPOINTSREQUEST_H_

#include "CEA2045MessageMacros.h"

namespace cea2045 {

struct cea2045SetSetpointsRequest
{
    unsigned char msgType1;
    unsigned char msgType2;
    unsigned short length;
    unsigned char opCode1;
    unsigned char opCode2;
    unsigned short deviceType;
    unsigned char units;
    unsigned short setpoint1;
    unsigned short setpoint2;
    unsigned short checksum;
};

}
```

```
void setDeviceType(unsigned short value)

{

    deviceType = htobe16(value);

}
```

```
void setSetpoint1(unsigned short value)

{

    setpoint1 = htobe16(value);

}
```

```
void setSetpoint2(unsigned short value)

{

    setpoint2 = htobe16(value);

}
```

```
MACRO_LENGTH
```

```
MACRO_CHECKSUM
```

```
} __attribute__((packed));
```

```
}
```

```
#endif /* CEA2045_CEA2045_MESSAGE_CEA2045MESSAGESETSETPOINTSREQUEST_H_ */
```

File: ./cea2045/cea2045/message/ConvertEnums.cpp

```
/*
```

```
* ConvertEnums.cpp
```

```
*
```


* Created on: Aug 24, 2015

* Author: dupes

*/

```
#include "../message/ConvertEnums.h"
```

```
#include <endian.h>
```

```
namespace cea2045 {
```

```
ConvertEnums::ConvertEnums()
```

```
{
```

```
}
```

```
ConvertEnums::~~ConvertEnums()
```

```
{
```

```
}
```

```
MessageTypeCode ConvertEnums::convertMessageType(unsigned short messageType)
```

```
{
```

```
    MessageTypeCode messageTypeCode = MessageTypeCode::NONE;
```

```
    switch (be16toh(messageType))
```

```
{
```

```
    case BASIC_MESSAGE:
```

```

    messageTypeCode = MessageTypeCode::BASIC;

    break;

case INTERMEDIATE_MESSAGE:

    messageTypeCode = MessageTypeCode::INTERMEDIATE;

    break;

case DATA_LINK_MESSAGE:

    messageTypeCode = MessageTypeCode::DATALINK;

    break;

default:

    messageTypeCode = MessageTypeCode::NONE;

}

return messageTypeCode;
}

LinkLayerNakCode ConvertEnums::convertLinkLayerNak(unsigned char nak)
{
    if (nak > (unsigned char)LinkLayerNakCode::REQUEST_NOT_SUPPORTED)

        return LinkLayerNakCode::RESERVED;

    return static_cast<LinkLayerNakCode>(nak);
}

```

```

DataLinkTypeCode ConvertEnums::convertDataLinkType(unsigned char dataLinkType)
{
    if (dataLinkType > (unsigned char)DataLinkTypeCode::SEND_NEXT_COMMAND_TO_SLOT)
        return DataLinkTypeCode::INVALID;

    return static_cast<DataLinkTypeCode>(dataLinkType);
}

```

```

MaxPayloadLengthCode          ConvertEnums::convertMaxPayloadLength(unsigned      char
maxPayloadLength)

{
    if (maxPayloadLength > (unsigned char)MaxPayloadLengthCode::LENGTH4096)
        return MaxPayloadLengthCode::INVALID;

    return static_cast<MaxPayloadLengthCode>(maxPayloadLength);
}

```

```

IntermediateTypeCode          ConvertEnums::convertIntermediateType(unsigned      short
intermediateType)

{
    intermediateType = be16toh(intermediateType);

    return static_cast<IntermediateTypeCode>(intermediateType);
}

```

```
} /* namespace cea2045 */
```

File: ./cea2045/cea2045/message/CEA2045MessageSetTemperatureOffsetRequest.h

```
/*
```

```
 * CEA2045MessageSetTemperatureOffsetRequest.h
```

```
 *
```

```
 * Created on: Apr 7, 2016
```

```
 * Author: dupes
```

```
 */
```

```
#ifndef CEA2045_CEA2045_MESSAGE_CEA2045MESSAGESETTEMPERATUREOFFSETREQUEST_H_
```

```
#define CEA2045_CEA2045_MESSAGE_CEA2045MESSAGESETTEMPERATUREOFFSETREQUEST_H_
```

```
#include "CEA2045MessageMacros.h"
```

```
namespace cea2045 {
```

```
struct cea2045SetTemperateOffsetRequest
```

```
{
```

```
    unsigned char msgType1;
```

```
    unsigned char msgType2;
```

```
    unsigned short length;
```

```
    unsigned char opCode1;
```

```
    unsigned char opCode2;
```

```
    unsigned char temperatureOffset;
```

```
    unsigned char units;
```

```
unsigned short checksum;

MACRO_LENGTH

MACRO_CHECKSUM

} __attribute__((packed));

}

#endif /* CEA2045_CEA2045_MESSAGE_CEA2045MESSAGESETTEMPERATUREOFFSETREQUEST_H_ */
```

File: ./cea2045/cea2045/message/CEA2045MessageGetSetpointsResponse.h

```
/*
 * CEA2045MessageGetSetpointsResponse.h
 *
 * Created on: Apr 25, 2016
 * Author: dupes
 */

#ifndef CEA2045_CEA2045_MESSAGE_CEA2045MESSAGEGETSETPOINTSRESPONSE_H_
#define CEA2045_CEA2045_MESSAGE_CEA2045MESSAGEGETSETPOINTSRESPONSE_H_

#include "CEA2045MessageMacros.h"

namespace cea2045 {

struct cea2045GetSetpointsResponse2

{
```

```
unsigned char msgType1;

unsigned char msgType2;

unsigned short length;

unsigned char opCode1;

unsigned char opCode2;

unsigned char responseCode;

unsigned short deviceType;

unsigned char units;

unsigned short setpoint1;

unsigned short setpoint2;

unsigned short checksum;
```

```
void setDeviceType(unsigned short value)

{

    deviceType = htobe16(value);

}
```

```
void setSetpoint1(unsigned short value)

{

    setpoint1 = htobe16(value);

}
```

```
void setSetpoint2(unsigned short value)

{

    setpoint2 = htobe16(value);

}
```

```

unsigned short getDeviceType()

{

    return be16toh(deviceType);

}


unsigned short getSetpoint1()

{

    return be16toh(setpoint1);

}


unsigned short getSetpoint2()

{

    return be16toh(setpoint2);

}


MACRO_LENGTH

MACRO_CHECKSUM

} __attribute__((packed));


/**

 * same as above but with only one setpoint; second setpoint is optional :/

 */

struct cea2045GetSetpointsResponse1

{

    unsigned char msgType1;

    unsigned char msgType2;

    unsigned short length;

```

```
unsigned char opCode1;

unsigned char opCode2;

unsigned char responseCode;

unsigned short deviceType;

unsigned char units;

unsigned short setpoint1;

unsigned short checksum;


void setDeviceType(unsigned short value)

{

    deviceType = htobe16(value);

}


void setSetpoint1(unsigned short value)

{

    setpoint1 = htobe16(value);

}


unsigned short getDeviceType()

{

    return be16toh(deviceType);

}


unsigned short getSetpoint1()

{

    return be16toh(setpoint1);

}
```



```
MACRO_LENGTH
```

```
MACRO_CHECKSUM
```

```
} __attribute__((packed));
```

```
}
```

```
#endif /* CEA2045_CEA2045_MESSAGE_CEA2045MESSAGEGETSETPOINTSRESPONSE_H_ */
```

File: ./cea2045/cea2045/message/CEA2045MessageBasic.h

```
/*
```

```
 * CEA2045MessageBasic.h
```

```
 *
```

```
 * Created on: Mar 5, 2015
```

```
 * Author: rgause
```

```
*/
```

```
#ifndef CEA2045MESSAGEBASIC_H_
```

```
#define CEA2045MESSAGEBASIC_H_
```

```
#include "CEA2045MessageMacros.h"
```

```
namespace cea2045 {
```

```
struct cea2045Basic
```

```
{
```

```
    unsigned char msgType1;
```

```
unsigned char msgType2;

unsigned short length;

unsigned char opCode1;

unsigned char opCode2;

unsigned short checksum;


MACRO_LENGTH

MACRO_CHECKSUM

} __attribute__((packed));


} // namespace cea2045


#endif /* CEA2045MESSAGEBASIC_H_ */
```

File: ../cea2045/cea2045/linklayer/ILinkLayerCommSend.h

```
/*

 * ILinkLayerSend.h

 *

 * Created on: Aug 25, 2015

 * Author: dupes

 */


#ifndef CEA2045_CEA2045_DEVICE_ILINKLAYERCOMMSSEND_H_

#define CEA2045_CEA2045_DEVICE_ILINKLAYERCOMMSSEND_H_


#include "../message/ConvertEnums.h"
```

```

namespace cea2045 {

class ILinkLayerCommSend

{

public:

    ILinkLayerCommSend() {};

    virtual ~ILinkLayerCommSend() {};


    virtual int sendRequest(const unsigned char *buffer, int length) = 0;


    virtual int sendResponse(const unsigned char *buffer, int length) = 0;


    virtual int sendLinkLayerAck() = 0;

    virtual int sendLinkLayerNak(LinkLayerNakCode nak) = 0;

};

} /* namespace cea2045 */


#endif /* CEA2045_CEA2045_DEVICE_ILINKLAYERCOMMSSEND_H_ */

```

File: ./cea2045/cea2045/linklayer/ILinkLayerCommRecv.cpp

```

/*

* ILinkLayerRecv.cpp

*

* Created on: Aug 25, 2015

* Author: dupes

*/

```

```
#include "ILinkLayerCommRecv.h"
```

```
namespace cea2045 {
```

```
ILinkLayerCommRecv::ILinkLayerCommRecv() {
```

```
}
```

```
ILinkLayerCommRecv::~ILinkLayerCommRecv() {
```

```
}
```

```
} /* namespace cea2045 */
```

File: ../cea2045/cea2045/linklayer/ReceiveBuffer.h

```
/*
```

```
 * ReceiveBuffer.h
```

```
 *
```

```
 * Created on: Aug 21, 2015
```

```
 * Author: dupes
```

```
 */
```

```
#ifndef CEA2045_CEA2045_RECEIVEBUFFER_H_
```

```
#define CEA2045_CEA2045_RECEIVEBUFFER_H_
```

```
#include "../communicationport/ICommunicationPort.h"
```

```

namespace cea2045 {

class ReceiveBuffer

{

private:

    unsigned char *m_buffer;


    unsigned int m_bufferLength;

    unsigned int m_numBytes;


public:

    ReceiveBuffer(unsigned int bufferLength);

    virtual ~ReceiveBuffer();


    int read(CommunicationPort *port, unsigned int waitMS);


    void reset();


    const unsigned char *getBuffer();

    unsigned int getNumBytes();

};

} /* namespace cea2045 */


#endif /* CEA2045_CEA2045_RECEIVEBUFFER_H_ */

```

File: ./cea2045/cea2045/linklayer/ReceiveBuffer.cpp

```
/*  
  
 * ReceiveBuffer.cpp  
  
 *  
  
 * Created on: Aug 21, 2015  
  
 * Author: dupes  
  
 */  
  
  
#include "ReceiveBuffer.h"  
  
  
namespace cea2045 {  
  
ReceiveBuffer::ReceiveBuffer(unsigned int bufferLength) :  
    m_bufferLength(bufferLength)  
{  
    m_buffer = new unsigned char[m_bufferLength];  
  
    m_numBytes = 0;  
}  
  
  
ReceiveBuffer::~~ReceiveBuffer()  
{  
    delete [] m_buffer;  
}  
  
  
int ReceiveBuffer::read(ICommunicationPort *port, unsigned int waitMS)
```

```
{  
  
    if (m_numBytes >= m_bufferLength)  
  
        return 0;  
  
  
  
    int bytesRead = port->recv(m_buffer + m_numBytes, m_bufferLength - m_numBytes, waitMS);  
  
  
  
    if (bytesRead > 0)  
  
        m_numBytes += bytesRead;  
  
  
  
    return bytesRead;  
  
}
```

```
void ReceiveBuffer::reset()  
  
{  
  
    m_numBytes = 0;  
  
}
```

```
const unsigned char *ReceiveBuffer::getBuffer()  
  
{  
  
    return m_buffer;  
  
}
```

```
unsigned int ReceiveBuffer::getNumBytes()  
  
{
```

```
    return m_numBytes;
}

} /* namespace cea2045 */
```

File: ../cea2045/cea2045/linklayer/ILinkLayerComm.h

```
/*
 * ILinkLayer.h
 *
 * Created on: Aug 25, 2015
 * Author: dupes
 */

#ifndef CEA2045_CEA2045_DEVICE_ILINKLAYERCOMM_H_
#define CEA2045_CEA2045_DEVICE_ILINKLAYERCOMM_H_

#include "ReceiveBuffer.h"

#include "../message/ConvertEnums.h"

#include "ILinkLayerCommRecv.h"

#include "ILinkLayerCommSend.h"

namespace cea2045 {

class ILinkLayerComm : public ILinkLayerCommSend, public ILinkLayerCommRecv
{
public:
```



```

    ILinkLayerComm() {};

    virtual ~ILinkLayerComm() {};

};

} /* namespace cea2045 */

#endif /* CEA2045_CEA2045_DEVICE_ILINKLAYERCOMM_H_ */

```

File: ./cea2045/cea2045/linklayer/ILinkLayerCommRecv.h

```

/*
 * ILinkLayerRecv.h
 *
 * Created on: Aug 25, 2015
 * Author: dupes
 */

#ifndef CEA2045_CEA2045_DEVICE_ILINKLAYERCOMMRECV_H_
#define CEA2045_CEA2045_DEVICE_ILINKLAYERCOMMRECV_H_

#include "ReceiveBuffer.h"

namespace cea2045 {

class ILinkLayerCommRecv
{
public:

    ILinkLayerCommRecv();

```

```

virtual ~ILinkLayerCommRecv();

virtual unsigned int recv(ReceiveBuffer &receiveBuffer, unsigned int waitMS) = 0;
};

} /* namespace cea2045 */

#endif /* CEA2045_CEA2045_DEVICE_ILINKLAYERCOMMRECV_H_ */

```

File: ./cea2045/cea2045/linklayer/LinkLayerCommImpl.h

```

/*
 * LinkLayerImpl.h
 *
 * Created on: Aug 25, 2015
 * Author: dupes
 */

#ifndef CEA2045_CEA2045_DEVICE_LINKLAYERCOMMIMPL_H_
#define CEA2045_CEA2045_DEVICE_LINKLAYERCOMMIMPL_H_

#include "../communicationport/ICommunicationPort.h"

#include "../util/MSTimer.h"

#include <mutex>

#include "ILinkLayerComm.h"

```

```

namespace cea2045 {

class LinkLayerCommImpl : public ILinkLayerComm
{
private:

    ICommunicationPort *m_communicationPort;

    MTimer m_tma;

    MTimer m_tar;

    std::mutex m_mutex;

    int send(const unsigned char *buffer, int length);

public:

    LinkLayerCommImpl(ICommunicationPort *communicationPort);

    virtual ~LinkLayerCommImpl();

    virtual int sendRequest(const unsigned char *buffer, int length);

    virtual int sendResponse(const unsigned char *buffer, int length);

    virtual int sendLinkLayerAck();

    virtual int sendLinkLayerNak(LinkLayerNakCode nak);

    virtual unsigned int recv(ReceiveBuffer &receiveBuffer, unsigned int waitMS);

};

```

```
} /* namespace cea2045 */
```

```
#endif /* CEA2045_CEA2045_DEVICE_LINKLAYERCOMMIMPL_H_ */
```

File: ./cea2045/cea2045/linklayer/LinkLayerCommImpl.cpp

```
/*
```

```
 * LinkLayerImpl.cpp
```

```
 *
```

```
 * Created on: Aug 25, 2015
```

```
 * Author: dupes
```

```
 */
```

```
#include "LinkLayerCommImpl.h"
```

```
#include "../message/CEA2045Message.h"
```

```
namespace cea2045 {
```

```
LinkLayerCommImpl::LinkLayerCommImpl(ICommunicationPort* communicationPort) :
```

```
    m_communicationPort(communicationPort),
```

```
    m_tma(40, false),
```

```
    m_tar(100, false)
```

```
{
```

```
}
```

```
LinkLayerCommImpl::~LinkLayerCommImpl()
```

```
{  
}
```

```
int LinkLayerCommImpl::send(const unsigned char* buffer, int length)
```

```
{  
    return m_communicationPort->send(buffer, length);  
}
```

```
int LinkLayerCommImpl::sendRequest(const unsigned char *buffer, int length)
```

```
{  
    return send(buffer, length);  
}
```

```
int LinkLayerCommImpl::LinkLayerCommImpl::sendResponse(const unsigned char *buffer, int  
length)
```

```
{  
    std::lock_guard<std::mutex> lock(m_mutex);
```

```
    m_tar.reset();
```

```
    m_tar.sleep();
```

```
    return send(buffer, length);  
}
```

```
int LinkLayerCommImpl::sendLinkLayerAck()
{
    std::lock_guard<std::mutex> lock(m_mutex);

    cea2045Link link;

    link.msgType1 = LINK_LAYER_ACK_MSG_TYP1;
    link.msgType2 = LINK_LAYER_ACK_MSG_TYP2;

    m_tma.reset();
    m_tma.sleep();

    return send((unsigned char*)&link, sizeof(link));
}
```

```
int LinkLayerCommImpl::sendLinkLayerNak(LinkLayerNakCode nak)
{
    std::lock_guard<std::mutex> lock(m_mutex);

    cea2045Link link;

    link.msgType1 = LINK_LAYER_NAK_MSG_TYP1;
    link.msgType2 = (unsigned char)nak;
}
```

```
m_tma.reset();

m_tma.sleep();


return send((unsigned char*)&link, sizeof(link));
}


unsigned int LinkLayerCommImpl::recv(ReceiveBuffer& receiveBuffer, unsigned int waitMS)
{
    return receiveBuffer.read(m_communicationPort, waitMS);
}


} /* namespace cea2045 */
```